



深圳大学

本科毕业论文（设计）

题目：面向工业机器人的嵌入式系统可靠固件更新架构设计

姓名：杨荣宝

专业：微电子科学与工程

学院：电子与信息工程

学号：2022280136

指导教师：刘春平

职称：教授

2026 年 05 月 12 日

深圳大学本科毕业论文（设计）诚信声明

本人郑重声明：所呈交的毕业论文（设计），题目《面向工业机器人的嵌入式系统可靠固件更新架构设计》是本人在指导教师的指导下，独立进行研究工作所取得的成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式注明。除此之外，本论文不包含任何其他个人或集体已经发表或撰写过的作品成果。本人完全意识到本声明的法律结果。

毕业论文（设计）作者签名：杨荣宝

2026 年 05 月 12 日

目 录

摘要	第 1 页
关键词	第 1 页
1 绪论	第 1 页
1.1 研究背景与意义	第 1 页
1.1.1 研究背景	第 1 页
1.1.2 研究意义	第 2 页
1.2 国内外研究现状	第 3 页
1.2.1 嵌入式 Bootloader 架构与 RTOS 应用现状	第 3 页
1.2.2 差分升级算法在资源受限设备中的研究现状	第 3 页
1.2.3 嵌入式底层启动机制与存储器管理优化现状	第 4 页
1.2.4 现有研究的局限性与本文的切入点	第 4 页
1.3 本文主要工作	第 5 页
1.3.1 存储空间映射重构与精细化内存管理	第 5 页
1.3.2 核心启动机制的纯 C 语言重写与代码复用	第 5 页
1.3.3 实时操作系统移植与系统资源监控设计	第 5 页
1.3.4 基于 Ymodem 协议的智能文件传输机制	第 6 页
1.3.5 轻量级差分还原算法 (dertools) 的移植与优化	第 6 页
1.3.6 本文的章节安排	第 6 页
2 系统总体架构与存储空间设计	第 6 页
2.1 系统总体架构设计	第 7 页
2.1.1 底层硬件抽象与启动层	第 7 页
2.1.2 存储与状态管理层	第 11 页
2.1.3 操作系统调度层	第 15 页
2.1.4 升级业务逻辑层	第 15 页
2.2 Flash 存储区划分与管理	第 15 页
2.2.1 LOADER 区 (引导与升级控制区)	第 16 页
2.2.2 USER 区 (用户应用程序区)	第 16 页
2.2.3 PATCH 区 (差分包暂存区)	第 17 页
2.2.4 OEM 区 (厂商出厂固件及基准备份区)	第 17 页
2.2.5 跨分区安全协同管理	第 17 页
2.3 RAM 存储区划分与数据共享机制	第 18 页
2.3.1 高速紧耦合内存 (ITCM 与 DTCM) 的加速应用	第 18 页

2.3.2	常规数据内存（AXI SRAM 与 AHB SRAM）的划分	第 18 页
2.3.3	启动参数区（Shared RAM）与数据共享机制	第 19 页
2.3.4	BKPRAM（备份 RAM）的持久化应用	第 19 页
3	核心底层机制与 RTOS 集成	第 20 页
3.1	基于 C 语言的启动文件深度重新设计	第 20 页
3.1.1	极简中断向量表设计与裸函数接管	第 22 页
3.1.2	底层时钟树与硬件权限的提前初始化	第 22 页
3.1.3	高效 C-Runtime (CRT) 的手动构建与循环展开优化	第 23 页
3.1.4	App 启动检测与多态跳转机制的实现	第 25 页
3.2	RT-Thread Nano 操作系统的移植与应用	第 26 页
3.2.1	底层架构适配	第 27 页
3.2.2	内存管理与系统裁剪	第 28 页
3.2.3	启动流程的重构与组件自动初始化	第 28 页
3.2.4	多任务解耦与业务调度机制	第 30 页
3.3	CPU 使用率监视器的设计与实现	第 31 页
3.3.1	测量原理与硬件选型	第 32 页
3.3.2	空闲任务钩子与睡眠时间统计	第 32 页
3.3.3	监视器线程与占用率计算	第 33 页
3.3.4	工程价值分析	第 33 页
4	固件传输协议与差分升级核心技术	第 34 页
4.1	基于 UART 与 Ymodem 协议的文件传输	第 34 页
4.1.1	Ymodem 协议的工作机制与 RTOS 适配	第 34 页
4.1.2	固件分类识别与智能路由机制	第 35 页
4.2	基于 detools 的差分还原算法实现	第 37 页
4.2.1	差分升级原理与 detools 选型优势	第 38 页
4.2.2	差分还原的流式处理（Streaming I/O）设计	第 38 页
4.2.3	安全的状态机与跳转控制	第 40 页
5	系统测试与分析	第 44 页
5.1	测试环境配置	第 44 页
5.2	启动与文件传输功能测试	第 47 页
5.2.1	引导程序与用户程序的双向跳转测试	第 47 页
5.2.2	Ymodem 大容量文件接收测试	第 50 页
5.3	差分升级性能对比测试	第 51 页
5.3.1	安全的状态机与跳转控制	第 51 页

5.4 CPU 占有率及内存开销评估	第 52 页
5.4.1 CPU 占用率分析	第 52 页
5.4.2 内存开销 (RAM/ROM) 评估	第 52 页
6 总结与展望	第 53 页
6.1 全文总结	第 53 页
6.2 后续工作展望	第 53 页
参考文献	第 55 页
致谢	第 57 页
Abstract	第 58 页
Key words	第 58 页

面向工业机器人的嵌入式系统可靠固件更新架构设计

电子与信息工程（微电子科学与工程）——杨荣宝

学号：2022280136

摘要

简述物联网及嵌入式设备对固件在线升级（OTA）的迫切需求。针对传统全量固件升级存在的传输时间长、Flash 擦写损耗大等问题，本文设计并实现了一款基于 32 位嵌入式单片机的 Bootloader 系统。该系统不仅支持标准的 UART 全量固件升级，还深度集成了 detools 差分还原算法实现高效的差分升级。系统通过重新划分 Flash 与 RAM 空间映射、采用纯 C 语言重新设计启动文件及复位逻辑，实现了 Loader 与 App 的无缝切换与代码复用。同时，引入 RT-Thread Nano 实时操作系统与 Ymodem 协议，提升了系统的任务调度能力和文件传输可靠性。最后，利用内核特性实现了 CPU 利用率监控。测试结果表明，该 Bootloader 运行稳定，差分升级功能显著降低了升级包体积与传输时间。

关键词

引导程序；差分升级；固件升级；实时操作系统

1 绪论

1.1 研究背景与意义

1.1.1 研究背景

随着物联网（IoT）、智能制造、机器人以及工业自动化技术的迅猛发展，嵌入式设备的应用场景日益复杂，设备数量呈爆发式增长。在产品的全生命周期中，为了修复系统漏洞、优化核心算法或迭代新功能，固件升级（Firmware Update）已成为嵌入式设备不可或缺的关键能力。^{[1][2]} 传统的固件更新通常依赖于物理烧录器（如 J-Link、ST-Link），这要求技术人员必须到现场拆卸设备外壳进行操作，维护成本极高且效率低下。^[3] 因此，基于通信接口（如 UART、以太网、无线网络等）的在应用编程（IAP, In-Application Programming）和在线升级（OTA, Over-the-Air）技术逐渐成为行业标准。^{[4][5]}

然而，在资源受限的 32 位嵌入式单片机（MCU）领域，现有的 Bootloader 设计与升级方案仍面临诸多痛点，亟待优化：

1. 全量升级带来的传输与存储瓶颈：传统的升级方式通常要求设备接收并全量重写整个新版本固件。^[1] 随着嵌入式软件业务逻辑的复杂化，固件体积不断膨胀

(通常在数百 KB 至数 MB 之间)。在 UART 等低带宽、易受干扰的通信接口下，全量固件传输耗时过长，不仅增加了升级过程被打断的风险（如突发断电导致设备“变砖”），还加剧了 Flash 存储器的擦写损耗。^[3]

2. 底层代码移植性差：主流的单片机启动文件（Startup File）通常由芯片厂商使用纯汇编语言编写，硬件耦合度极高。^[6]当项目需要跨芯片平台移植，或希望在 Bootloader 与应用程序（App）之间实现底层初始化代码的复用，繁杂的汇编指令往往成为技术壁垒。^{[7] [8]}
3. 裸机框架的并发处理能力弱：传统的 Bootloader 多采用裸机（前后台系统）轮询的架构。^[9]在面对大文件接收（如 Ymodem 协议解析）、复杂的数据解压算法^[10]以及系统状态监控时，单死的死循环结构容易导致处理阻塞，文件传输的稳定性与系统的实时响应能力捉襟见肘。^{[8] [11]}

1.1.2 研究意义

针对上述行业痛点，本文设计并实现了一款基于 32 位嵌入式 MCU 的差分升级 Bootloader 系统。本课题的研究不仅在底层架构上进行了深度创新，更在实际工程应用中具有显著的经济与技术价值，具体体现在以下几个方面：^[7]

1. 突破带宽限制，大幅提升升级效率（差分升级的价值）

本项目将 detools 差分还原算法引入资源受限的 MCU 端。通过在 PC 端对比新旧固件生成体积极小的差分包（Patch），设备端仅需通过 UART 接收差分包即可在本地还原出全量新固件。^{[1] [2]}这种方式不仅将数据传输量降低了约 70%~90%，极大缩短了升级时间，继而显著降低了升级失败的概率，特别适用于受限于低速串口通信的工业现场设备。^{[12] [10]}

2. 重塑存储模型，增强系统健壮性（空间规划的价值）

本文打破了常规的 Flash 和 RAM 粗放式分配，进行了精细化的空间重构。在 Flash 端划分出独立的 PATCH 区与 OEM 区，确保在差分还原过程中，即使发生意外中断，原有用户程序（USER 区）与出厂固件也不受破坏；在 RAM 端，创新性地规划了“启动参数区”。利用该区域在软件复位时数据不丢失的特性，实现了 Bootloader 与 App 之间状态标志与命令的安全、无缝传递，大幅提升了程序跳转的确定性与系统的容错率。^{[8] [9]}

3. 提高代码复用率与跨平台可移植性（C 语言重写启动文件的价值）^[6]

摒弃了晦涩的汇编启动代码，本文深度采用纯 C 语言重新设计了 reset_handler 复位逻辑及 RAM 数据初始化（如 .data 段搬运和 .bss 段清零）。这一底层重构不仅让 Bootloader 具备了极强的跨硬件平台移植能力^[9]，还使得 Bootloader 与 App 可以共享同一套初始化架构，大幅降低了工程代码的冗余度与后期维护成本。^[7]

4. 引入实时操作系统，提升资源调度与监控能力（RTOS 的价值）

通过在 Bootloader 中移植 RT-Thread Nano 实时操作系统，将串口接收（Ymodem 协议处理^[11]）、差分数据解压还原、CPU 状态监控等功能解耦为独立的线程。^[9] 多任务并发调度彻底解决了裸机架构下的阻塞问题，不仅保证了文件传输的极高可靠性，还能通过实时输出的 CPU 利用率监控数据，为系统性能评估与算法内存优化提供直观的量化指标。^[13]

综上所述，本文所设计的 Bootloader 系统，在保证极高可靠性的前提下，实现了固件升级技术的“轻量化”与“高效化”。研究成果不仅为 32 位嵌入式 MCU 的 OTA 升级提供了一套高内聚、低耦合的标准模板，也对降低物联网设备的后期运维成本具有重要的工程指导意义。

1.2 国内外研究现状

随着物联网产业的规模化发展，嵌入式设备的在线升级（OTA, Over-the-Air）与在应用编程（IAP, In-Application Programming）技术已成为国内外学术界与工业界的研究热点。^[4] ^[1] Bootloader 作为实现固件升级的核心底层软件，其架构设计、传输协议及更新算法经历了从简单到复杂、从粗放式到精细化的演进过程。当前国内外的研究现状主要集中在以下几个维度：^[7] ^[2]

1.2.1 嵌入式 Bootloader 架构与 RTOS 应用现状

早期的 MCU Bootloader 通常采用简单的“裸机”前后台架构（即 while(1) 超级循环加中断）。^[9] 这种架构虽然对 Flash 和 RAM 资源占用极小，但在处理复杂逻辑时存在明显短板。例如，在通过 UART 接收文件时，如果同时需要进行大面积的 Flash 擦写操作，极易引发系统的阻塞，导致通信协议（如 Xmodem、Ymodem 等）超时报错甚至升级失败。^[11]

近年来，随着 32 位 MCU（如 ARM Cortex-M 系列）硬件资源的不断丰富，国内外许多学者开始致力于将轻量级实时操作系统（RTOS）引入 Bootloader 中。^[9] 例如，FreeRTOS、国内的 RT-Thread Nano 等被广泛应用于升级程序的底层调度。^[13] 通过多线程管理，系统能够将“通信接收”、“固件存储”、“状态监控”等任务解耦。研究表明，在 Bootloader 中引入 RTOS，不仅能大幅提升 Ymodem 等文件传输协议的稳定性和容错率，还能利用操作系统的空闲任务（Idle Task）实现对 CPU 占用率的精确监控，为系统资源的性能评估提供数据支撑。^[7] ^[14] ^[15]

1.2.2 差分升级算法在资源受限设备中的研究现状

传统的固件升级方式多为“全量升级”，即每次都下载完整的固件包。这在网络带宽有限或对功耗要求极高的场景下（如 NB-IoT、LoRa 或低速 UART 总线设备）表现出极大的局限性。^[16] ^[17] 因此，差分升级技术（Differential Update）逐渐由 PC 端和智能手机端（如 Android 系统的 OTA）下沉至嵌入式领域。^[3]

在 PC 和 Linux 嵌入式系统中，主流的差分算法是基于 `bsdiff/bspatch` 或 `xdelta`。^[2]
^[10] 然而，这些经典算法在还原阶段通常依赖 `bzip2` 解压机制，需要占用极大的动态内存（通常在 MB 级别），这对于 RAM 空间仅有几十到上百 KB 的常规 32 位单片机而言是完全无法承受的。^{[16] [17]}

针对这一痛点，近年来国内外工业界开源了多种适用于微控制器的轻量化差分算法。其中，`dertools` 由于其极低的内存开销（RAM Footprint）和优异的压缩比，成为了研究的热点。^[2] `dertools` 结合 `crle`（一种基于游程编码的无损数据压缩算法）等机制，支持流式处理（Streaming I/O）。^[17] 现有的先进研究主要致力于将这类算法与 MCU 的底层 Flash 读写深度结合，通过在 Flash 中划分出专门的 PATCH 暂存区，使得 MCU 可以在不将完整固件载入 RAM 的情况下，边读取、边解压、边写入，成功解决了小内存运行复杂差分算法的业界难题。^{[12] [10]}

1.2.3 嵌入式底层启动机制与存储器管理优化现状

Bootloader 的设计深度依赖于 MCU 的底层启动机制和存储器映射。^[7] 传统上，绝大多数芯片厂商（如 ST、NXP）提供的启动文件（Startup File）均采用汇编语言编写。^{[8] [6]} 这种做法虽然执行效率高，但代码晦涩难懂，且与特定的编译器（如 Keil MDK、IAR、GCC）强绑定，导致跨平台移植困难。^[18] 同时，Bootloader 和 App 之间难以实现底层初始化代码的复用。^[15]

为了提高代码的可维护性和跨平台能力，国内外部分前沿开源项目和学术研究开始探索“纯 C 语言启动框架”。通过在 C 语言层面接管复位中断（`reset_handler`），手动控制 `.data` 段的搬运和 `.bss` 段的清零，不仅消除了对汇编的依赖，还使得 Loader 阶段能够灵活控制外设的初始化状态。

此外，在存储器管理方面，传统的升级参数传递多采用将标志位写入 Flash 特定扇区的方式，这会增加 Flash 的擦写损耗并延长启动时间。当前的优化研究倾向于利用 RAM 的精细化划分：一方面利用 MCU 的高速紧耦合内存（ITCM/DTCM）加速核心算法的执行；另一方面，开辟出不被复位清零的“启动参数区（No-Init RAM）”和 BKPRAM 备份区。通过软复位（Soft Reset）技术在内存中直接传递跳转指令和状态标识，极大地提高了 Bootloader 与 App 切换的响应速度与系统健壮性。^[7]

1.2.4 现有研究的局限性与本文的切入点

综合来看，虽然国内外在嵌入式 OTA 升级、RTOS 应用以及轻量级差分算法上均取得了丰硕的成果，但在实际工程应用中，能够将这些先进技术进行深度整合与极致优化的方案依然较少。^[16] 大部分方案要么仅停留在全量升级阶段，要么其差分升级仍面临内存溢出的风险；此外，底层启动代码的僵化仍然制约着 Bootloader 代码的跨项目复用。^[3]

基于上述现状，本文以解决传统升级方案中“传输慢、移植难、阻塞卡顿”等痛点为切入点，深入研究并设计了一款集成了 RT-Thread Nano 多任务调度^[13]、Ymodem 可靠传输^[11] 以及 `dertools` 差分核心算法的现代 Bootloader。^{[16] [17]} 在此基础上，本文创新

性地重新设计了 C 语言底层启动文件，并对存储器空间进行了精细化重构，旨在为 32 位嵌入式设备提供一套高可靠、低开销的差分升级完整解决方案。^[8]

1.3 本文主要工作

本文针对传统 32 位嵌入式设备固件升级中存在的传输效率低、底层框架耦合度高、大文件接收易阻塞等工程痛点，设计并实现了一款具备差分升级能力的高可靠 Bootloader 系统。^[15] 本项目不仅在应用层实现了高效的协议与算法，还在底层对单片机的启动机制与存储器分配进行了深度重构。本文的主要研究与开发工作包括以下五个方面：^[7]

1.3.1 存储空间映射重构与精细化内存管理

打破了传统 MCU 粗放的存储器划分方式，重新设计了 Flash 与 RAM 的空间布局。在 Flash 侧，严格划分出 LOADER 区（引导程序）、USER 区（用户程序）、PATCH 区（差分包暂存）及 OEM 区（出厂固件备份），保障了升级过程的物理安全性；在 RAM 侧，充分利用芯片内核特性划分了 ITCM 与 DTCM 以加速核心算法执行，并创新性地开辟了“启动参数区（No-Init RAM）”。该区域的数据在软件复位（Soft Reset）时不会丢失，成功实现了 Bootloader 与 App 之间升级指令与状态标志的无缝传递。^{[19] [15]}

1.3.2 核心启动机制的纯 C 语言重写与代码复用

摒弃了传统高度依赖芯片厂商的汇编启动文件（Startup File），通过纯 C 语言深度重构了系统的启动流程。独立编写了 reset_handler 复位处理逻辑，并在 C 语言层面接管了 .data 段的搬运与 .bss 段的清零等 RAM 初始化工作。这一设计不仅大幅提升了 Bootloader 代码的跨平台可移植性，还使得 Bootloader 与 User App 能够复用同一套底层初始化框架。同时，在复位处理函数中加入了严格的 App 启动检测机制（如栈顶指针合法性校验、复位向量校验等），确保程序跳转的安全性。^{[7] [15]}

1.3.3 实时操作系统移植与系统资源监控设计

为解决裸机轮询架构在面对大文件接收和复杂解压算法时的阻塞问题，本文在 Bootloader 中成功移植了轻量级实时操作系统 RT-Thread Nano。^[13] 通过引入多任务调度机制，将业务逻辑模块化分离为串口接收、差分还原、系统监控等独立线程，显著提升了系统的并发处理能力。此外，利用 ARM Cortex-M 内核的周期计数器等硬件特性，设计并实现了一套 CPU 使用率监视器，能够实时统计并输出系统负载，为后续算法的性能调优提供了量化依据。^[14]

1.3.4 基于 Ymodem 协议的智能文件传输机制

在 UART 通信基础上移植了 Ymodem 文件传输协议，实现了稳定可靠的固件接收状态机。^{[18] [11]} 针对全量升级与差分升级并存的需求，设计了基于文件首包（Header Packet）的智能解析分类机制。系统通过提取下发文件的名称或后缀特征，自动判别固件类型：若为全量固件，则直接触发全量更新流程；若为差分包（Patch），则将其安全存入 PATCH 区，并触发差分还原流水线。

1.3.5 轻量级差分还原算法（detools）的移植与优化

将适用于资源受限设备的轻量化差分工具 detools 深度集成至 MCU 端。^[12] 配合 PC 端使用 crle 算法生成的极小体积差分包，在设备端实现了高效的固件还原计算。针对 MCU 内部 RAM 极小的物理限制，采用了流式处理（Streaming I/O）策略，使得系统无需将整个固件或差分包载入内存，即可通过“边读、边解压、边写”的方式，利用 OEM 区（或当前 USER 区）的旧版固件作为基准，重构出完整的新版固件，彻底解决了单片机差分升级的内存溢出瓶颈。

1.3.6 本文的章节安排

为了清晰地阐述上述研究工作，本文的具体章节安排如下：

1. 绪论：阐述课题的研究背景、意义，分析当前国内外固件升级技术的研究现状，并概括本文的主要研究内容。
2. 系统总体架构与存储空间设计：详细论述 Bootloader 的系统拓扑结构，以及 Flash 和 RAM 多重区域的精细化划分与功能定义。
3. 核心底层机制与 RTOS 集成：重点剖析基于纯 C 语言重新设计启动文件与复位逻辑的技术细节，并介绍 RT-Thread Nano 的移植过程及 CPU 利用率监视器的实现原理。
4. 固件传输协议与差分升级核心技术：深入讲解 Ymodem 协议状态机的实现与文件分类逻辑，详细推演基于 detools 与 crle 算法的差分还原流程及内存优化策略。
5. 系统测试与分析：搭建测试环境，对 Bootloader 的启动切换、全量/差分传输成功率、CPU 与内存开销、升级耗时等关键指标进行系统性测试与量化分析。
6. 总结与展望：总结全文研究成果，客观分析当前系统的不足，并对未来的技术迭代与扩展方向进行展望。

2 系统总体架构与存储空间设计

本章主要详细论述存储器（Memory）的重新规划，这是 Bootloader 运行的基础。

2.1 系统总体架构设计

为了实现高可靠、低耦合的差分固件升级，并充分发挥 32 位 MCU（本项目采用带有复杂存储器架构的 Cortex-M7 内核，如 STM32H743IIT6）的硬件性能，本系统摒弃了传统的“单体大循环”裸机架构，采用了分层解耦的模块化架构设计。

系统总体架构自下而上可划分为四个核心层次：底层硬件抽象与启动层、存储与状态管理层、操作系统调度层以及升级业务逻辑层。

2.1.1 底层硬件抽象与启动层

该层负责在编码阶段尽量屏蔽硬件差异，在编译阶段提供 Bootloader 与 App 统一的链接模板脚本，以及系统上电后的“第一条指令”执行与底层硬件运行环境的初始化。

- 链接脚本模板：通过不同的全局编译宏定义生成适用于 Loader 和 App 的链接脚本，二者的 Flash 与 RAM 分区布局相同，避免 App 固件起始地址错乱。链接脚本模板：[19] [4] [8] [15]

```
#include "partition.h"

/**
 * @brief 定义flash与ram。
 */
MEMORY
{
    /* flash */
    LOADER (rwx) : ORIGIN = LOADER_START,      LENGTH = LOADER_SIZE
    USER   (rwx) : ORIGIN = USER_START,        LENGTH = USER_SIZE
    PATCH  (rw)  : ORIGIN = PATCH_START,        LENGTH = PATCH_SIZE
    OEM     (rwx) : ORIGIN = OEM_START,         LENGTH = OEM_SIZE

    /* tcm */
    ITCM    (rwx) : ORIGIN = MCU_ITCM_START,    LENGTH = MCU_ITCM_SIZE
    DTCM    (rwx) : ORIGIN = MCU_DTCM_START,    LENGTH = MCU_DTCM_SIZE

    /* sram */
    AXIRAM  (rwx) : ORIGIN = MCU_AXIRAM_START,  LENGTH = MCU_AXIRAM_SIZE
    AHBRAM  (rwx) : ORIGIN = MCU_AHBRAM_START,  LENGTH = MCU_AHBRAM_SIZE
    AHBRAM1 (rwx) : ORIGIN = MCU_AHBRAM1_START, LENGTH = MCU_AHBRAM1_SIZE
    BKPRAM  (rwx) : ORIGIN = MCU_BKPRAM_START,  LENGTH = MCU_BKPRAM_SIZE
}

/* 初始栈地址(从高地址向底地址生长) */
_stack_start = ORIGIN(DTCM) + LENGTH(DTCM);

/* 栈最小大小 */
_stack_min_size = 0x400;

/* 堆结束地址(紧靠栈且不覆盖最小栈空间) */
_heap_end = _stack_start - _stack_min_size;
```

```

/* 根据构建参数匹配不同的段分配布局 */
#ifdef BUILD_LOADER
#define CODE_PARTITION LOADER
#elif BUILD_USER
#define CODE_PARTITION USER
#elif BUILD_OEM
#define CODE_PARTITION OEM
#else
/* 无有效参数时报错 */
#error "BUILD_LOADER, BUILD_USER or BUILD_OEM must be defined"
#endif

/**
 * @brief 定义段如何分配。
 */
SECTIONS
{
    /* 普通代码段 */
    .code : ALIGN(4)
    {
        KEEP(*(.reset_vector)) /* 复位向量表 */
        *(.text) /* 普通代码 */
        *(.glue_7*) /* ARM/Thumb胶水代码 */
        KEEP(*(.rt_launch_run.*)) /* os启动时自动执行 */
    } > CODE_PARTITION

    /* 快速代码段 */
    .itcm : ALIGN(4)
    {
        _itcm_ram_start = .; /* 起始地址 */
        KEEP(*(.itcm*)) /* 快速代码 */
        _itcm_ram_end = .; /* 结束地址 */
    } > ITCM AT > CODE_PARTITION
    _itcm_section_addr = LOADADDR(.itcm);

    /* 常量数据段 */
    .const : ALIGN(4)
    {
        *(.rodata*) /* 常量数据 */
    } > CODE_PARTITION

    /* DTCM已初始化数据段 */
    .dtcm_init : ALIGN(4)
    {
        _dtcm_ram_init_start = .; /* 起始地址 */
        *(.data*) /* 已初始化数据 */
        _dtcm_ram_init_end = .; /* 结束地址 */
    } > DTCM AT > CODE_PARTITION
    _dtcm_ram_section_addr = LOADADDR(.dtcm_init);

    /* DTCM未初始化数据段 */
    .dtcm_uninit : ALIGN(4)
    {

```

```

        _dtcm_ram_uninit_start = .;          /* 起始地址 */
        *(.bss*)                  /* 未初始化数据 */
        _dtcm_ram_uninit_end = .;          /* 结束地址 */
    } > DTCM

/* 堆空间段 */
.heap : ALIGN(8)
{
    _heap_start = .;                /* 紧跟在数据段之后 */
} > DTCM

/* 栈空间段 */
.stack : ALIGN(8)
{
    . = . + _stack_min_size;        /* 不足以分配最小栈空间时报错 */
} > DTCM

/* AXIRAM已初始化数据段 */
.axi_ram_init : ALIGN(4)
{
    _axi_ram_init_start = .;        /* 起始地址 */
    _axi_ram_init_end = .;         /* 结束地址 */
} > AXIRAM AT > CODE_PARTITION
_axi_ram_section_addr = LOADADDR(.axi_ram_init);

/* AXIRAM未初始化数据段 */
.axi_ram_uninit : ALIGN(4)
{
    _axi_ram_uninit_start = .;      /* 起始地址 */
    _axi_ram_uninit_end = .;       /* 结束地址 */
} > AXIRAM

/* AHBRAM已初始化数据段 */
.ahb_ram_init : ALIGN(4)
{
    _ahb_ram_init_start = .;        /* 起始地址 */
    *(.share_init)                  /* 已初始化数据 */
    _ahb_ram_init_end = .;         /* 结束地址 */
} > AHBRAM AT > CODE_PARTITION
_ahb_ram_section_addr = LOADADDR(.ahb_ram_init);

/* AHBRAM未初始化数据段 */
.ahb_ram_uninit : ALIGN(4)
{
    _ahb_ram_uninit_start = .;      /* 起始地址 */
    *(.share_uninit)                /* 未初始化数据 */
    _ahb_ram_uninit_end = .;       /* 结束地址 */
} > AHBRAM

/* share数据段 */
.share_ram : ALIGN(4)
{

```

```

        _share_ram_start = .;           /* 起始地址 */
        *(.share)           /* 程序之间共享的数据 */
        _share_ram_end = .;           /* 结束地址 */
    } > AHBRAM1 AT > CODE_PARTITION
    _share_ram_section_addr = LOADADDR(.share_ram);

    /* BKPRAM已初始化数据段 */
    .bkp_ram : ALIGN(4)
    {
        _bkp_ram_start = .;           /* 起始地址 */
        *(.backup)           /* 备份数据 */
        _bkp_ram_end = .;           /* 结束地址 */
    } > BKPRAM

    /* 显式丢弃的内容 */
    /DISCARD/ :
    {
        /* 丢弃C++异常回溯表 */
        *(.ARM.extab* .gnu.linkonce.armextab.*)
        *(.ARM.exidx*)

        /* 丢弃编译器自带的TLS占位符 */
        *(.tdata*)
        *(.tbss*)

        /* 丢弃C标准库和数学库 */
        libc.a:* (*)
        libm.a:* (*)
    }
}

```

然后通过 GCC 编译器进行预处理后得到对应 Loader 与 App 的链接脚本

▸ Loader 指令脚本（EIDE 插件环境变量）：

```

${CompilerPrefix}gcc.exe -E -P -x c -DBUILD_LOADER ${workspaceFolder}/bsp/
include/mcu_port.ld -o ${OutDir}/diffboot.ld

```

▸ App 指令脚本（EIDE 插件环境变量）：

```

${CompilerPrefix}gcc.exe -E -P -x c -DBUILD_APP ${workspaceFolder}/bsp/
include/mcu_port.ld -o ${OutDir}/diffboot.ld

```

- 向量表与复位接管：在精简的汇编文件（vector.s）中定义复位向量表（vector_table），并将系统复位入口直接映射到纯 C 语言实现的 reset_handler()。
- 硬件内核初始化：在 reset_handler() 中，系统优先调用 fpu_init() 开启浮点运算单元执行权限，调用 rcc_init() 配置系统时钟树（PLL）和 Flash 延迟（Latency），并调用 ram_init() 使能 MCU 特有的多个高级 RAM 域（如 D2SRAM、BKPRAM 及其独立时钟）。

- C 运行环境（C-Runtime）手动构建：系统利用内联优化的 C 函数 `reset_copy_ram_init()` 和 `reset_clear_ram_uninit()`，将 Flash 中的初始数据精确地搬运至 ITCM、DTCM、AXIRAM 及 AHBRAM 等指定的物理内存区域，并完成 .bss 段的清零，为操作系统的运行和差分算法的高效计算（如利用 ITCM 存放高频代码，DTCM 存放高频数据）做好了物理铺垫。

2.1.2 存储与状态管理层

该层是打通 Bootloader（引导程序）与 User App（用户程序）的信息桥梁，负责物理空间的边界划分与运行状态的安全传递。^{[7] [15]}

- 物理存储分区映射：系统通过 `partition.h` 实现了 Flash 的四等分布局（LOADER 区、USER 区、PATCH 区、OEM 区），严格定义了各区间的地址（如 `MCU_FLASH_START` 为 `0x08000000`）与扇区容量，隔离了核心代码与下载数据的存储空间。具体实现：^{[19] [4]}

```
#ifndef _PARTITION_H_
#define _PARTITION_H_

#include "specification.h" //!< 导入内存划分与段分配

/**
 * @brief 各分区占用扇区数量。
 */
#define LOADER_SECTOR_COUNT 2 //!< 引导程序占用扇区数量
#define USER_SECTOR_COUNT 6 //!< 用户程序占用扇区数量
#define PATCH_SECTOR_COUNT 2 //!< 差分补丁占用扇区数量
#define OEM_SECTOR_COUNT 6 //!< 厂商程序占用扇区数量

/**
 * @brief 内存各分区占用大小。
 */
#define LOADER_SIZE (LOADER_SECTOR_COUNT * MCU_FLASH_SECTOR_SIZE) //!< 引导程序占用大小
#define USER_SIZE (USER_SECTOR_COUNT * MCU_FLASH_SECTOR_SIZE) //!< 用户程序占用大小
#define PATCH_SIZE (PATCH_SECTOR_COUNT * MCU_FLASH_SECTOR_SIZE) //!< 差分补丁占用大小
#define OEM_SIZE (OEM_SECTOR_COUNT * MCU_FLASH_SECTOR_SIZE) //!< 厂商程序占用大小

/**
 * @brief 各分区起始地址。
 */
#define LOADER_START MCU_FLASH_START //!< 引导程序起始地址
#define USER_START (LOADER_START + LOADER_SIZE) //!< 用户程序起始地址
#define PATCH_START (USER_START + USER_SIZE) //!< 差分补丁起始地址
#define OEM_START (PATCH_START + PATCH_SIZE) //!< 厂商程序起始地址
```

```
#endif
```

- 共享状态参数区 (Shared RAM): 这是架构设计的一大亮点。系统通过自定义的编译器属性宏 (SECTION(“.share")), 将一个名为 load_config 的状态结构体 (包含错误码、复位标志、升级策略、差分包大小等信息) 显式分配到“启动参数区”。具体实现: [20] [21]

```
#ifndef _ATTRIBUTE_H_
#define _ATTRIBUTE_H_

/**
 * @brief 指定函数为弱符号, 实现可被覆盖。
 */
#define WEAK __attribute__((weak))

/**
 * @brief 避免函数因优化而剔除。
 */
#define USED __attribute__((used))

/**
 * @brief 明确指定不使用该函数。
 */
#define UNUSE_FUNC __attribute__((unused))

/**
 * @brief 明确指定不使用该变量。
 */
#define UNUSE_VAR(x) ((void)x)

/**
 * @brief 禁止编译器自动给函数添加任何汇编代码。
 */
#define NAKED __attribute__((naked))

/**
 * @brief 强制内联函数。
 */
#define INLINE __attribute__((always_inline))

/**
 * @brief 消除结构体成员之间的填充, 实现紧凑存储。
 */
#define PACKED __attribute__((packed))

/**
 * @brief 起始地址进行n字节对齐。
 */
#define ALIGN(n) __attribute__((aligned(n)))

/**
```

```

* @brief 添加段分配属性。
*/
#define SECTION(name) __attribute__((section(name)))

/**
* @brief 不添加任何属性。
*/
#define NONE

/**
* @brief 将函数放在itcm运行，提高代码执行速度或者避免与flash操作冲突。
*/
#define ITCM SECTION(".itcm")

/**
* @brief 将数据放在boot与app共享的ram，
* boot与app之间的切换不会影响这些数据。
*/
#define SHARE SECTION(".share")

/**
* @brief 将有值数据放在外设可访问的ram，
* boot与app之间的切换会覆盖这些数据。
*/
#define SHARE_INIT SECTION(".share_init")

/**
* @brief 将无值数据放在外设可访问的ram，
* boot与app之间的切换可能会覆盖这些数据。
*/
#define SHARE_UNINIT SECTION(".share_uninit")

/**
* @brief 将数据放在备份ram，电池供电时始终保持数据存在，
* 除非程序的显示修改。
*/
#define BACKUP SECTION(".backup")

#endif

```

```

/**
* @brief 定义错误选项。
*/
typedef enum load_error_t {
    LOAD_ERROR_VERIFY = 0x00, //!< 数据验证无效
    LOAD_ERROR_WHICH = 0x01, //!< 启动参数错误
    LOAD_ERROR_INVALID = 0xff, //!< 无效参数
} load_error_t;

/**
* @brief 定义复位选项。
*/
typedef enum load_reset_t {

```

```

    LOAD_RESET          = 0x00, //!< 需要执行复位
    LOAD_RESET_INVALID = 0xff, //!< 无效参数
} load_reset_t;

/**
 * @brief 定义启动选项。
 */
typedef enum load_which_t {
    LOAD_APP_LOADER = 0x00, //!< 从loader启动
    LOAD_APP_USER   = 0x01, //!< 从user启动
    LOAD_APP_OEM    = 0x02, //!< 从oem启动
    LOAD_APP_INVALID = 0xff, //!< 无效参数
} load_which_t;

/**
 * @brief 定义打补丁选项。
 */
typedef enum load_apply_t {
    LOAD_APPLY_USER   = 0x00, //!< 应用补丁生成新固件到user
    LOAD_APPLY_OEM    = 0x01, //!< 应用补丁生成新固件到oem
    LOAD_APPLY_INVALID = 0xff, //!< 无效参数
} load_apply_t;

/**
 * @brief 定义固件接收选项。
 */
typedef enum load_patch_t {
    LOAD_PATCH_USER   = 0x00, //!< 接收到user的补丁
    LOAD_PATCH_OEM    = 0x01, //!< 接收到oem的补丁
    LOAD_PATCH_INVALID = 0xff, //!< 无效参数
} load_patch_t;

/**
 * @brief 定义如何启动程序。
 */
typedef struct load_config_info_t {
    volatile load_error_t error;  //!< 错误信息
    volatile load_reset_t reset;  //!< 是否需要执行复位
    volatile load_which_t which;  //!< 从哪个程序启动
    volatile load_apply_t apply;  //!< 生成新固件到哪个分区
    volatile load_patch_t patch;  //!< 接收到哪个补丁
    volatile uint32_t patch_size; //!< 补丁大小
} load_config_info_t;

/**
 * @brief 定义如何校验配置。
 */
typedef struct load_config_t {
    load_config_info_t info; //!< 配置信息
    volatile uint16_t crc;   //!< crc校验码
} load_config_t;

```

- 状态机校验与跳转控制：load.c 模块实现了基于 CRC16 的强校验机制^[22] (algo_crc16)。在软复位时，Loader 通过 load_read_config_which() 读取 load_config。^[21] 若校验通过且目标明确，系统通过重写栈指针 (__set_MSP)、更新中断向量表偏移 (SCB->VTOR)，执行向 USER 区或 OEM 区的安全跳转；若不满足跳转条件，则留在 Loader 层，并向下调用 load_boot() 准备初始化升级组件。^[20]

2.1.3 操作系统调度层

作为承上启下的中枢，该层通过引入 RT-Thread Nano 实时操作系统，彻底改变了 Bootloader 的执行流。

- OS 接管与启动：在 load_boot() 函数中，系统完成了基础外设配置后，通过 rtthread_launch() 正式移交 CPU 控制权。具体实现：^[8]

```
void load_boot(void)
{
    main();           //!< 配置时钟树与外设
    rtthread_launch(); //!< 启动rtthread
}
```

- 多任务并发与资源隔离：操作系统将文件接收、差分还原、状态监控等功能拆分为独立的线程，利用操作系统的调度器 (Scheduler) 和信号量 (Semaphore) 机制，解决了大体量差分固件处理时可能导致的阻塞卡顿问题。^[14]

2.1.4 升级业务逻辑层

处于架构最顶端，包含了面向用户的两大核心业务组件：

- 通信与文件传输组件：在多线程支持下，运行 Ymodem 协议状态机^[23]，稳定地通过 UART 接收上位机下发的 .bin (全量) 或 .patch (差分) 文件，并基于文件特征将其流式写入 Flash 的对应分区 (USER 区或 PATCH 区)。
- 差分还原引擎：集成了 detools 和 crle 解压算法。^[12] 当检测到 PATCH 区有有效的差分包时，引擎利用 OEM 区 (或当前 USER 区) 作为基准 (Base)，在有限的 RAM 空间内进行“流式运算” (Streaming I/O)，将合成后的新固件安全覆盖到 USER 区，最终修改 load_config 状态并触发 MCU 重启以加载新固件。

2.2 Flash 存储区划分与管理

在嵌入式设备的在线固件升级 (OTA) 过程中，Flash 存储器的物理划分方案直接决定了系统的安全性、升级成功率以及存储资源的利用率。本项目所采用的 MCU 为 STM32H743IIT6，其内部片上 Flash 总容量为 2MB，物理擦除粒度 (扇区大小 MCU_FLASH_SECTOR_SIZE) 为 128KB (0x00020000)，共计 16 个扇区。

传统的 Bootloader 通常仅划分“Boot 区”与“App 区”，在升级时直接覆写 App 区。这种做法一旦在升级过程中发生掉电或通信中断，极易导致设备固件损坏且无法恢复（即设备“变砖”）。为了满足差分升级算法（detools）的运行需求，并提供极高的防掉电容错能力，本文在 partition.h 中打破了常规布局，将 2MB 的 Flash 空间精细化地重构为四个逻辑分区：LOADER 区、USER 区、PATCH 区和 OEM 区。具体划分与管理机制如下：^[19]

地址范围	分区名称与功能	容量 / 扇区数量
0x081FFFFFF 0x08140000	OEM 区 (厂商出厂固件)	768 KB (6 个扇区)
0x0813FFFF 0x08100000	PATCH 区 (差分包暂存)	256 KB (2 个扇区)
0x080FFFFFF 0x08040000	USER 区 (用户程序)	768 KB (6 个扇区)
0x0803FFFF 0x08000000	LOADER 区 (引导程序)	256 KB (2 个扇区)

表 1: STM32H743IIT6 Flash 存储器物理空间划分表

2.2.1 LOADER 区 (引导与升级控制区)

- 空间规划：分配 2 个扇区（共 256KB），起始地址为 0x08000000。
- 核心功能：本区域是系统上电复位后最先执行的代码段。内部固化了底层时钟树配置、C 语言运行环境构建、RT-Thread Nano 操作系统核心、Ymodem 串口文件传输协议以及 detools 差分还原引擎。^{[13] [23]}
- 设计考量：虽然基础的 Bootloader 通常仅需几十 KB，但由于本项目集成了完整的 RTOS 和复杂的流式解压缩算法，因此预留了 256KB 的充足空间，保证了后续 Bootloader 自身业务逻辑的扩展性。^[7] 为了确保系统永远具备可恢复能力，在任何升级流程中，LOADER 区均被设置为写保护，严禁被擦除或修改。^[19]

2.2.2 USER 区 (用户应用程序区)

- 空间规划：分配 6 个扇区（共 768KB），起始地址紧随 LOADER 区之后（0x08040000）。
- 核心功能：用于存放设备在正常工况下运行的实际业务逻辑代码（即 User App）。

- 管理机制：在引导阶段，当 load.c 模块读取到“启动参数区”的启动标识为 LOAD_APP_USER，且校验合法时，Bootloader 会关闭自身中断，将系统中断向量表偏移寄存器（SCB->VTOR）重定向至 0x08040000，并更新主栈指针（MSP），最终通过函数指针跳转至该区域执行。在全量升级或差分还原的最后阶段，新固件的数据将被写入此区域。^{[19] [21]}

2.2.3 PATCH 区（差分包暂存区）

- 空间规划：分配 2 个扇区（共 256KB），起始地址为 0x08100000。
- 核心功能：专门用于暂存通过 UART 接收到的差分升级包（Patch 文件）。
- 设计考量（核心亮点）：这是支撑差分升级的灵魂分区。在常规全量升级中，暂存区需要与 App 区等大（即也需要 768KB 的空间），这在资源紧张的 MCU 中往往难以实现（常被称为“A/B 双区升级的存储瓶颈”）。而本文引入差分技术后，得益于 crle 算法极高的压缩比，新旧版本之间生成的差分包体积通常不足原固件大小的 30%。因此，分配 256KB 作为 PATCH 区已存在巨大的冗余量，完美解决了小容量 Flash 实现安全 OTA 的痛点。通过暂存机制，避免了在下载阶段直接破坏正在运行的旧版固件。^[19]

2.2.4 OEM 区（厂商出厂固件及基准备份区）

- 空间规划：分配 6 个扇区（共 768KB），与 USER 区大小严格保持一致，起始地址为 0x08140000。
- 核心功能：存放设备出厂时烧录的稳定版本固件（Base Firmware）。
- 管理机制：OEM 区在系统中扮演双重角色：^[19]
 1. 差分还原基准：当差分引擎对 PATCH 区的压缩包进行解压时，需要提取旧数据参与运算。系统可将 OEM 区的稳定固件作为固定的还原基准源流出，与 PATCH 包通过 detools 算法合成出全新的固件并写入 USER 区。^[12]
 2. 出厂恢复（Fallback）：如 load.c 代码所示，系统设计了 LOAD_APP_OEM 启动分支。当 USER 区因非法断电、恶意篡改或新版固件存在致命 Bug 导致无法运行时，Bootloader 能够侦测到异常，并在下一次重启时强制将 SCB->VTOR 导向 OEM 区，实现系统的“一键恢复出厂设置”，极大提升了工业现场设备的健壮性。

2.2.5 跨分区安全协同管理

通过上述物理划分，系统构建了一个高度解耦的存储器管理模型。在一次完整的差分升级任务中，Flash 管理流程如下：

1. 传输阶段：Bootloader 将接收到的 .patch 数据流式写入 PATCH 区，此过程完全不影响 USER 区与 OEM 区的数据完整性。^[7]

2. 合成阶段：系统从 OEM 区（或当前运行正常的 USER 区）读取基准字节，从 PATCH 区读取差分指令与补丁字节，两者在内存中交汇运算后，将生成的新固件逐扇区擦除并写入 USER 区。
3. 收尾阶段：写入完成并进行 CRC 校验后，更新系统级标志位，完成向 USER 区的安全切换。^[21]

该分区模型严格遵循了“隔离更新、留有退路”的高可靠软件设计原则，结合 STM32H7 大扇区的硬件特性，在节省 Flash 擦写寿命与提升升级容错率之间取得了完美的平衡。

2.3 RAM 存储区划分与数据共享机制

与传统的单总线单片机不同，现代高性能 32 位 MCU（如基于 Cortex-M7 内核的 STM32H7 系列）内部拥有多块挂载在不同总线上的独立 RAM 域。为了最大化利用硬件性能，加速 detools 差分还原算法的执行，并解决跨程序（Bootloader 与 App）状态传递的工程难题，本文对 MCU 的内部 RAM 进行了深度的精细化划分与属性分配。系统整体 RAM 架构被划分为以下四大功能区：^[12]

2.3.1 高速紧耦合内存（ITCM 与 DTCM）的加速应用

Cortex-M7 架构特有的紧耦合内存（TCM）与处理器内核直接相连，能够实现指令和数据的“零等待状态（Zero-wait state）”访问，其运行速度远超挂载在 AXI 或 AHB 体系上的普通 SRAM。

- ITCM（指令紧耦合内存）：系统在 specification.h 中分配了近 64KB 的 ITCM 空间（起始地址 0x00000000）。为了防止与内核复位中断向量表（Vector Table）发生地址冲突，代码中巧妙地设置了 0x400 的起始偏移量。在工程应用中，通过自定义属性宏 SECTION(“.itcm”)，将 Flash 擦写核心函数以及差分解压算法（如 crle 核心逻辑）映射至此。系统复位时，底层 C 代码（reset_copy_ram_init）会主动将这些关键代码从 Flash 搬运至 ITCM 中执行。此举不仅大幅提升了差分还原的解算帧率，还彻底避免了“在 Flash 中擦写 Flash”时产生的总线访问冲突（Read-while-Write Stall）问题。
- DTCM（数据紧耦合内存）：分配了 128KB 空间（起始地址 0x20000000）。通过链接脚本控制，该区域主要用于存放 RT-Thread Nano 的系统堆栈、高频调度的全局变量以及中断上下文中频繁访问的数据，从而将操作系统的调度开销降至最低。^[13]

2.3.2 常规数据内存（AXI SRAM 与 AHB SRAM）的划分

STM32H7 拥有容量庞大的 AXI SRAM（512KB）和多块 AHB SRAM（如 288KB 的 SRAM1/2 和 64KB 的 SRAM4）。

在本系统中，这些常规 RAM 区域主要被作为大数据量的吞吐缓冲区。例如，在 Ymodem 协议接收大文件或 detools 进行流式读写（Streaming I/O）时，需要分配大量动态内存（Heap）作为缓存队列。^[12] 系统在 reset.c 的 reset_handler 函数中，通过 C 语言接管了这些内存域的有值数据搬运（reset_copy_ram_init）和无值数据清零（reset_clear_ram_uninit），确保了文件系统的缓存读写操作不会受到未初始化脏数据的影响。

2.3.3 启动参数区 (Shared RAM) 与数据共享机制

在固件升级的完整闭环中，“App 通知 Loader 有新包待更新”以及“Loader 升级结束后跳转回 App”是两个最关键的切换动作。传统的方案通常将状态标志写入 Flash 的特定扇区，但这种做法会引发擦写耗时并加剧 Flash 物理磨损。

为了实现零损耗、高实时性的状态传递，本文设计了基于“**No-Init RAM**（非初始化内存）”的共享数据机制：

1. 内存属性定义：在 attribute.h 中定义了宏 SECTION(“.share”)，配合链接脚本，在 RAM 中强行开辟了一段独立区域，并在底层启动文件（reset.c）的初始化流中刻意绕开了对该区域的清零操作。这意味着，只要芯片未发生物理掉电（即仅通过软件指令触发的 Soft Reset），该区域的数据将持续驻留。
2. 数据结构与防篡改机制：在 load.c 中定义了共享结构体 load_config_t，其中封装了启动目标（LOAD_APP_USER / LOAD_APP_OEM）、错误码（如 LOAD_ERROR_VERIFY）、差分包大小及操作标志。为了防止上电瞬间 RAM 数据随机性带来的误触发，系统为该结构体引入了严格的 algo_crc16 循环冗余校验。^[20]
3. 状态机流转：当用户在 App 中触发下载并暂存了 .patch 差分包后，App 会修改 load_config_t 的标志位并更新 CRC16 校验值，随后触发 NVIC_SystemReset()（软件复位）。系统重启进入 Loader 后，会第一时间读取并校验该共享结构体，若 CRC 匹配且指令合法，则直接启动升级状态机；升级完成后，Loader 再次修改该共享参数，重定向至 USER 区并触发跳转。^[21]

2.3.4 BKPRAM (备份 RAM) 的持久化应用

针对特殊应用场景，系统还在 specification.h 中启用了 4KB 的 BKPRAM（备份 RAM，起始地址 0x38800000）。

在 ram_init() 初始化流中，系统使能了电源控制寄存器（PWR->CR1 |= PWR_CR1_DBP）的后备域访问权限。该区域挂载在 Vbat 后备电池域下，即使主电源完全断电，其数据也不会丢失。通过 SECTION(“.backup”) 宏修饰，该区域被用于保存极其关键的系统级日志（如升级中断的历史记录或设备防篡改时间戳），进一步提升了设备在恶劣工业现场环境下的抗掉电追溯能力。

3 核心底层机制与 RTOS 集成

对于一款支持复杂差分算法与高可靠在线升级的 Bootloader 而言，底层的运行机制决定了系统的上限。传统的裸机开发模式高度依赖芯片厂商预编译的汇编启动代码，且在面对大体量固件传输和高耗时解压算法时，容易出现系统阻塞。针对这一痛点，本文在底层机制上进行了深度重构。本章首先详细阐述了基于纯 C 语言重写的 MCU 启动文件与复位流程，随后介绍了轻量级实时操作系统 RT-Thread Nano 的集成过程，最后论述了如何利用内核硬件特性构建精确的 CPU 使用率监控系统^[14]，从而为多任务调度与升级算法的稳定运行提供坚实的底层保障。^[13]

3.1 基于 C 语言的启动文件深度重新设计

在 ARM Cortex-M 架构的传统开发中，启动文件（如 startup_stm32h743xx.s）通常由数百行晦涩的汇编代码构成，负责初始化堆栈、清零 BSS 段、搬运 DATA 段以及调用 SystemInit 和 main 函数。这种高度硬件耦合的做法极大地限制了代码的跨平台可移植性，且导致 Bootloader 与 User App 之间难以共享底层初始化逻辑。

为此，本文摒弃了常规的纯汇编启动架构，采用“极简汇编向量表 + 纯 C 语言复位逻辑”的混合架构，对系统的启动（reset_handler）与 C 运行环境（C-Runtime）构建进行了彻底的深度重写。

- 原始汇编启动流程：

```
.section .text.Reset_Handler
.weak Reset_Handler
.type Reset_Handler, %function
Reset_Handler:
    ldr sp, =_estack /* set stack pointer */

/* Call the ExitRun0Mode function to configure the power supply */
    bl ExitRun0Mode
/* Call the clock system initialization function.*/
    bl SystemInit

/* Copy the data segment initializers from flash to SRAM */
    ldr r0, =_sdata
    ldr r1, =_edata
    ldr r2, =_sidata
    movs r3, #0
    b LoopCopyDataInit

CopyDataInit:
    ldr r4, [r2, r3]
    str r4, [r0, r3]
    adds r3, r3, #4

LoopCopyDataInit:
    adds r4, r0, r3
    cmp r4, r1
```

```

    bcc CopyDataInit
/* Zero fill the bss segment. */
ldr r2, =_sbss
ldr r4, =_ebss
movs r3, #0
b LoopFillZerobss

FillZerobss:
str r3, [r2]
adds r2, r2, #4

LoopFillZerobss:
cmp r2, r4
bcc FillZerobss

/* Call static constructors */
bl __libc_init_array
/* Call the application's entry point.*/
bl main
bx lr
.size Reset_Handler, .-Reset_Handler

```

- 深度重新设计后启动流程:

```

NAKED void reset_handler(void)
{
    // 设置初始栈指针
    __set_MSP((uint32_t)_stack_start); //!< 执行后可以正常使用栈空间

    // 调用初始化函数
    fpu_init(); //!< 开启fpu执行权限
    rcc_init(); //!< 配置默认的系统时钟树
    ram_init(); //!< 开启所有ram区域

    /* 加载itcm的数据 */
    reset_copy_ram_init((char*)_itcm_ram_start, _itcm_section_addr,
                        (size_t)_itcm_ram_end -
                        (size_t)_itcm_ram_start); //!< 拷贝itcm的数据

    #if defined(BUILD_LOADER)
        // 尝试启动app程序
        load_app(); //!< 所有app不满足启动要求则返回并开始启动boot
    #endif

    /* 加载dttcm的数据 */
    reset_copy_ram_init(
        (char*)_dttcm_ram_init_start, _dttcm_ram_section_addr,
        (size_t)_dttcm_ram_init_end -
        (size_t)_dttcm_ram_init_start); //!< 拷贝dttcm的有值数据
    reset_clear_ram_uninit(
        (char*)_dttcm_ram_uninit_start,
        (char*)_dttcm_ram_uninit_end); //!< 清空dttcm的无值数据

```

```

/* 加载axiram的数据 */
reset_copy_ram_init(
    (char *)_axi_ram_init_start, _axi_ram_section_addr,
    (size_t)_axi_ram_init_end -
        (size_t)_axi_ram_init_start); //!< 拷贝axiram的有值数据
reset_clear_ram_uninit(
    (char *)_axi_ram_uninit_start,
    (char *)_axi_ram_uninit_end); //!< 清空axiram的无值数据

/* 加载ahbram的数据 */
reset_copy_ram_init(
    (char *)_ahb_ram_init_start, _ahb_ram_section_addr,
    (size_t)_ahb_ram_init_end -
        (size_t)_ahb_ram_init_start); //!< 拷贝ahbram的有值数据
reset_clear_ram_uninit(
    (char *)_ahb_ram_uninit_start,
    (char *)_ahb_ram_uninit_end); //!< 清空ahbram的无值数据

load_boot(); //!< 程序数据已经加载完成，开始启动boot程序

default_handler(); //!< 正常不会执行到这里，进入默认处理函数，执行死循环
}

```

3.1.1 极简中断向量表设计与裸函数接管

为了将控制权平滑地移交给 C 语言，系统在 `vector.s` 中仅保留了最基础的中断向量表（`vector_table`）结构声明。向量表的首个字定义为初始堆栈指针（`_stack_start`），第二个字即指向了复位处理程序（`reset_handler`）。

在 `reset.c` 模块中，本文利用 GCC 编译器的 `__attribute__((naked))` 属性宏（在系统代码中封装为 `NAKED` 宏），定义了纯 C 语言的复位处理函数 `reset_handler(void)`。`NAKED` 属性能够阻止编译器自动插入函数序言（Prologue）和跋语（Epilogue），确保复位后的第一条指令完全受控。进入该函数后，系统第一时间调用 `__set_MSP()` 强行刷新主栈指针，确保后续的 C 语言函数调用拥有安全的压栈空间。

3.1.2 底层时钟树与硬件权限的提前初始化

在进入 C 语言运行环境之前，必须确保关键硬件域的时钟已经就绪。在重写的 `reset_handler` 中，系统依次调用了底层硬件初始化接口：

1. FPU 权限开启（`fpu_init`）：通过配置系统控制块协处理器访问控制寄存器（`SCB->CPACR`），提前使能浮点运算单元（FPU），以支持后续差分算法中可能涉及的高效运算。
2. 时钟树配置（`rcc_init`）：重新配置 Flash 访问延迟（Latency）并配置系统锁相环（PLL），使得 CPU 尽早进入最高主频运行状态，缩短系统启动耗时。

3. 高级 RAM 域使能 (ram_init): 针对 STM32H7 系列多总线 RAM 的特性, 主动开启 D2SRAM1/2/3 等高级内存域的总线时钟 (RCC_AHB2ENR), 并使能 BKPRAM 的电源后备域访问权限, 为后续的数据搬运铺平物理通道。

3.1.3 高效 C-Runtime (CRT) 的手动构建与循环展开优化

由于剥离了标准库的启动文件, 本文在 C 语言中手动实现了原本由编译器隐式完成的 C-Runtime 环境构建。包括对全局变量初始值的搬运 (.data 段处理) 和未初始化变量的清零 (.bss 段处理)。

值得注意的是, 针对 STM32H7 庞大的内存域划分 (ITCM、DTCM、AXIRAM、AHBRAM), 为了将数据从相对较慢的 Flash 搬运至不同总线的 RAM 中, 本文在 reset.h 中设计了两个极具效率的内联函数: reset_copy_ram_init() 和 reset_clear_ram_uninit()。

- 内存对齐与 32 位宽优化: 通过位运算 (((uintptr_t)dest & 0x3) != 0) 对非对齐地址进行字节级修补, 在地址对齐到 4 字节边界后, 立刻切换为 32 位 (uint32_t) 指针进行宽字读写, 将总线利用率提升了 4 倍。
- 循环展开优化 (Loop Unrolling): 在此基础上, 运用了激进的“循环展开”算法。在处理大量数据时 (while (len >= 16)), 一次循环体内部直接连续处理 4 个 32 位字 (即 16 字节)。这种写法大幅度降低了 CPU 在大内存块拷贝时由于循环跳转带来的流水线冲刷 (Pipeline Flush) 和条件分支开销, 显著缩短了 detools 差分解压缓冲区庞大内存的初始化时间。^[12]
- 具体实现:

```
/**
 * @brief 将数据从flash拷贝到ram。
 * @param dest 拷贝数据到该地址。
 * @param src 从该地址拷贝数据。
 * @param len 拷贝字节数。
 */
INLINE static inline char *reset_copy_ram_init(char *dest, const char *src,
                                                size_t len)
{
    // 小数据量处理
    if (len < 4) {
        while (len-- > 0) *dest++ = *src++;
        return dest;
    }

    // 对齐处理 (尝试将目标地址对齐到4字节边界)
    // 检查地址是否对齐到4字节
    while (((uintptr_t)dest & 0x3) != 0 && len > 0) {
        *dest++ = *src++;
        len--;
    }

    // 32位宽数据拷贝 (核心优化)
```

```

uint32_t *d32      = (uint32_t *)dest;
const uint32_t *s32 = (const uint32_t *)src;

// 使用循环展开：一次处理16字节（4个32位字）
// 这样可以减少循环次数和判断开销
while (len >= 16) {
    d32[0] = s32[0];
    d32[1] = s32[1];
    d32[2] = s32[2];
    d32[3] = s32[3];
    d32 += 4;
    s32 += 4;
    len -= 16;
}

// 处理剩余的32位字
while (len >= 4) {
    *d32++ = *s32++;
    len -= 4;
}

// 处理剩余的字节
dest = (char *)d32;
src  = (const char *)s32;
while (len--) { *dest++ = *src++; }

return dest;
}

```

```

/**
 * @brief 将ram指定区域清零。
 * @param start 起始地址。
 * @param end   结束地址。
 */
INLINE static inline char *reset_clear_ram_uninit(char *start, char *end)
{
    // 将8位字符扩展为32位模式
    // 将c转换为无符号8位，然后填充到32位变量中
    uint8_t val8  = 0u;
    uint32_t val32 = (uint32_t)val8;
    val32 |= (val32 << 8);
    val32 |= (val32 << 16);
    uint32_t n = (uint32_t)end - (uint32_t)start;

    // 字节对齐前的填充（处理起始不齐的部分）
    while (((uintptr_t)start & 0x3) != 0 && n > 0) {
        *start++ = val8;
        n--;
    }

    // 32位宽批量填充（核心优化）
    uint32_t *p32 = (uint32_t *)start;

```

```

// 循环展开：一次填充16字节
while (n >= 16) {
    p32[0] = val32;
    p32[1] = val32;
    p32[2] = val32;
    p32[3] = val32;
    p32 += 4;
    n -= 16;
}

// 处理剩余的32位字
while (n >= 4) {
    *p32++ = val32;
    n -= 4;
}

// 处理最后的剩余字节
start = (char *)p32;
while (n--) { *start++ = val8; }

return start;
}

```

3.1.4 App 启动检测与多态跳转机制的实现

在纯 C 环境构建完成后，Bootloader 需要决定自身的运行去向，本文将其设计为“预检测跳转”机制。

在 reset_handler 中，紧随 ITCM 代码搬运之后，系统调用了 load.c 模块中的 load_app() 接口：

1. 状态机判定：系统读取基于共享 RAM（Shared RAM）的 load_config 结构体，通过比对启动标识（LOAD_APP_USER 或 LOAD_APP_OEM），判断当前是否满足跳转用户程序的条件。
2. 多态复位向量提取：若满足跳转条件，系统从对应的 Flash 分区（USER_START 或 OEM_START）提取该分区的初始堆栈指针与复位函数向量。
3. 无缝移交控制权：通过 __disable_irq() 关闭全局中断，修改系统中断向量表偏移寄存器（SCB->VTOR）重定向中断映射，刷新堆栈指针，最终通过强转的函数指针（new_reset_handler()）跨物理分区唤醒用户程序或厂商固件。具体实现：^[18]

```

void load_app(void)
{
    load_clear_error(); //!< 清除复位前设置的错误码
    load_clear_reset(); //!< 清除复位前设置的复位需求

    uint32_t app_bin_addr; //!< 待启动的app程序地址
    load_which_t which;
    if (!load_read_config_which(&which))

```

```

{
    return;
}
switch (which) //!< 读取应该启动哪个程序
{
case LOAD_APP_LOADER:
    return; //!< 从boot启动
case LOAD_APP_USER:
    app_bin_addr = USER_START; //!< 从用户程序启动
    break;
case LOAD_APP_OEM:
    app_bin_addr = OEM_START; //!< 从厂商程序启动
    break;
default:
    load_set_error(Load_Error_Which);
    return; //!< 无效参数时不加载app程序
}

load_write_config_which(Load_App_Invalid); //!< 清除启动配置

const uint32_t new_msp =
    *(volatile uint32_t *)app_bin_addr; //!< 获取app的栈指针
const void_fn_void_t new_reset_handler = (void_fn_void_t)(*(
    volatile uint32_t *)app_bin_addr + 4)); //!< 获取app的复位处理函数

__disable_irq();                //!< 防止在跳转过程中被中断打断
SCB->VTOR = app_bin_addr;        //!< 设置向量表偏移
__set_MSP(new_msp);              //!< 更新到app的初始栈指针
new_reset_handler();             //!< 执行跳转
}

```

若校验失败或当前存在升级任务，函数直接返回，系统继续执行剩余 RAM 区的数据加载，并最终进入 `load_boot()`，启动 RT-Thread 操作系统接管控制权。^{[21] [13]} 这种 C 语言层面的模块化接管，不仅实现了 Bootloader 对 App 跳转的强控，更由于 C 框架的通用性，使得该工程代码可作为标准模块直接移植到其他 Cortex-M 芯片项目中实现底层复用。

3.2 RT-Thread Nano 操作系统的移植与应用

在传统的 Bootloader 设计中，通常采用裸机（Bare-metal）前后台轮询的架构。然而，随着差分升级（涉及复杂的 Flash 擦写与解压算法计算）与 Ymodem 通信协议的引入，裸机架构容易导致阻塞，影响系统的实时性与稳定性。为此，本文在 Bootloader 中引入了极其轻量级的实时操作系统 RT-Thread Nano，以实现复杂业务的解耦。^[13]

3.2.1 底层架构适配

针对本项目所采用的基于 ARM Cortex-M7 内核的单片机（STM32H743），系统的底层移植主要涵盖三个方面：

1. 系统心跳（Tick）配置：在 board.c 中，将 SysTick 定时器配置为 1000Hz（即每秒触发 1000 次中断），在 SysTick_Handler 中断服务函数中调用 rt_tick_increase()，为操作系统提供精准的时基，具体实现。

```
/**
 * @brief 系统滴答定时器中断处理。
 */
ITCM void SysTick_Handler(void)
{
    rt_interrupt_enter();
    HAL_IncTick();
    rt_tick_increase();
    rt_interrupt_leave();
}

/**
 * @brief 精确微秒延时实现。
 * @param us 延时长度(微秒)。
 * @warning 请勿执行过长延时!
 * @note 关中断与阻塞式实现!
 */
ITCM void rt_hw_us_delay(uint32_t us)
{
    __disable_irq();

    const uint32_t ticks = us * ticks_per_us;
    const uint32_t start = DWT->CYCCNT;

    // 循环等待直到达到目标周期数
    // 无符号减法会自动处理0xFFFFFFFF到0的翻转
    while ((DWT->CYCCNT - start) < ticks)
        ;

    __enable_irq();
}
```

2. 适配微秒级延时：

```
/**
 * @brief 精确微秒延时实现。
 * @param us 延时长度(微秒)。
 * @warning 请勿执行过长延时!
 * @note 关中断与阻塞式实现!
 */
ITCM void rt_hw_us_delay(uint32_t us)
{
    __disable_irq();
```

```

const uint32_t ticks = us * ticks_per_us;
const uint32_t start = DWT->CYCCNT;

// 循环等待直到达到目标周期数
// 无符号减法会自动处理0xFFFFFFFF到0的翻转
while ((DWT->CYCCNT - start) < ticks)
    ;

__enable_irq();
}

```

3.2.2 内存管理与系统裁剪

由于 Bootloader 的 RAM 资源受限（需为用户 App 预留大部分空间），必须对 RTOS 进行深度裁剪：

- 配置精简：在 `rtconfig.h` 中，关闭了设备虚拟文件系统（VFS）、软件定时器（Software Timer）等不必要组件。仅保留了信号量、互斥锁和邮箱（Mailbox）等核心 IPC（进程间通信）机制。
- 堆内存接管：开启了小内存管理算法（`RT_USING_SMALL_MEM_AS_HEAP`）。利用链接脚本（Linker Script）导出的 `_heap_start` 与 `_heap_end` 符号，在 `rt_hw_mcu_init` 阶段调用 `rt_system_heap_init`，将紧跟在数据段之后的常规 RAM 区域交给 RTOS 接管，使得系统能够使用 `rt_malloc` 和 `rt_free` 动态分配 Ymodem 接收缓冲区和差分算法的计算内存。具体实现：^[23]

```

/* 堆结束地址(紧靠栈且不覆盖最小栈空间) */
_heap_end = _stack_start - _stack_min_size;

/* 堆空间段 */
.heap : ALIGN(8)
{
    _heap_start = .;          /* 紧跟在数据段之后 */
} > DTCM

```

```

#ifdef RT_USING_HEAP
    // 初始化rtos堆内存
    rt_system_heap_init((void *)_heap_start, (void *)_heap_end);
    LOG_V("heap: [0x%p, 0x%p]", _heap_start, _heap_end);
#endif

```

3.2.3 启动流程的重构与组件自动初始化

系统调用 `rtthread_launch()` 后，不仅完成了 MCU 外设时钟的初始化（`rt_hw_mcu_init`），还引入了自动初始化机制。通过自定义的段属性宏（如 `RT_LAUNCH_RUN_EXPORT`），将各个外设模块和系统组件的初始化函数指针集中存

放到特定的 Flash 段中。^[8] 在 `rt_hw_board_init` 阶段，通过遍历该指针数组依次执行初始化，实现了 Bootloader 业务模块间的彻底解耦。具体实现：^[13]

```
/**
 * @brief 执行操作系统启动前的配置。
 */
void rt_hw_mcu_init(void)
{
    // MCU复位启动完成
    LOG_F("mcu reset finish");

    // 更新MCU内核时钟
    LOG_V("cpu clock per s: %u", HAL_RCC_GetSysClockFreq());

    // 计算1us对应的时钟周期数
    ticks_per_us = HAL_RCC_GetSysClockFreq() / 1000000;
    LOG_V("tick per us: %u", ticks_per_us);

    // 初始化内核计数器
    bool result = rt_hw_dwt_init();
    if (!result)
    {
        LOG_E("dwt init fail");
    }
    else
    {
        LOG_I("dwt init success");
    }

    // 配置微秒级延时
    LL_LPTIM_Enable(LPTIM1);
    LL_LPTIM_SetAutoReload(LPTIM1, 0xFFFF); // 设置最大计数值
    LL_LPTIM_StartCounter(LPTIM1, LL_LPTIM_OPERATING_MODE_CONTINUOUS);
    LOG_I("start lptim counter");

    // 配置systick中断频率
    HAL_SYSTICK_Config(HAL_RCC_GetSysClockFreq() / RT_TICK_PER_SECOND);
    LOG_V("systick per s: %u", RT_TICK_PER_SECOND);

#ifdef RT_USING_HEAP
    // 初始化rtos堆内存
    rt_system_heap_init((void *)_heap_start, (void *)_heap_end);
    LOG_V("heap: [0x%p, 0x%p]", _heap_start, _heap_end);
#endif

    // 设置空闲钩子
    rt_thread_idle_sethook(idle_hook_wfi);
    LOG_I("add idle hook: idle_hook_wfi");
}
```

3.2.4 多任务解耦与业务调度机制

依托 RTOS 的调度能力，系统将复杂的 OTA 流程划分为多个独立线程：

- **Boot 守护线程：**作为主控线程（优先级较高），负责轮询检测 `load_config` 标志位。当接收到差分包后，调用 `detools_apply_patch` 进行固件还原，并执行软复位（`NVIC_SystemReset`）。具体实现：^{[12] [13]}

```
/**
 * @brief bootloader线程。
 * @param parameter 线程名称字符串。
 */
static void boot_thread_entry(void *parameter)
{
    #if defined(BUILD_LOADER)
        LOG_D("launch from boot");
    #elif defined(BUILD_USER)
        LOG_D("launch from user");
    #elif defined(BUILD_OEM)
        LOG_D("launch from oem");
    #else
        LOG_F("lost BUILD_XXX define");
    #endif
    while (1)
    {
        LOG_D("<thread:%s> running", parameter);
        HAL_GPIO_TogglePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin);
        detect_apply();
        detect_reset();
        rt_thread_mdelay(500);
    }
}
```

- **Ymodem 通信线程：**专门负责监听 UART 端口。由于 RTOS 提供了阻塞挂起机制，该线程在没有串口数据时会出让 CPU 权限，不再占用死循环。具体实现：^{[18] [13] [23]}

```
/**
 * @brief ymodem线程。
 * @param parameter 线程名称参数。
 */
static void ymodem_thread_entry(void *parameter)
{
    while (1)
    {
        // 实际应用中，可在此处判断是否进入下载模式
        ymodem_receive_loop();
    }
}
```

- **系统监视线程：**用于监控系统健康状态和资源占用（详见 3.3 节）。具体实现：^{[13] [14]}

```

/**
 * @brief 监视器线程。
 * @param parameter 线程名称字符串。
 */
static void monitor_thread_entry(void *parameter)
{
    // 将毫秒转换为系统的Tick数
    const rt_tick_t period_tick = rt_tick_from_millisecond(1000);

    // 获取进入循环前的当前Tick时间作为基准
    rt_tick_t last_wakeup_tick = rt_tick_get();

    while (1)
    {
        /* 取出数据并清零 */
        rt_enter_critical();
        uint64_t sleep_cnt = rt_idle_total_sleep_get();
        rt_idle_total_sleep_clear();
        rt_exit_critical();

        /* 理论上1秒钟LPTIM应该走1000000个Tick */
        if (sleep_cnt > 1000000)
        {
            sleep_cnt = 1000000; // 修正误差，防止出现负数
        }

        static uint8_t cnt = 0;
        if (++cnt % 1 == 0)
        {
            /* 占用率 = 100 - (睡眠占比) */
            LOG_D("cpu usage per s: %.2f%%",
                  100.0f - ((float)sleep_cnt / 1000000) * 100.0f);
            cnt = 0;
        }

        /*
         * 阻塞延时至下一个绝对时间点
         * 内核会自动计算：需要sleep多久 = (last_wakeup_tick + period_tick) -
         * current_tick 并自动更新 last_wakeup_tick
         */
        rt_thread_delay_until(&last_wakeup_tick, period_tick);
    }
}

```

3.3 CPU 使用率监视器的设计与实现

在差分升级过程中，解压还原算法（detools/crle）属于 CPU 密集型任务，同时 Flash 擦写又属于 I/O 耗时型操作。^[12] 为了精准评估 Bootloader 运行时的系统负载，为后续算法的内存与速度 trade-off（权衡）优化提供数据支撑，本文基于 RT-Thread Idle Hook（空闲钩子）机制与 MCU 硬件定时器，设计并实现了一套高精度的 CPU 使用率监视器。^[13]

3.3.1 测量原理与硬件选型

常规的 CPU 使用率统计通常依赖操作系统的 Tick 计数，但由于 Tick 精度仅为毫秒级（1ms），且在系统进入休眠（Sleep）时容易出现统计误差，无法满足微观层面的性能分析需求。

本文巧妙利用了 STM32H7 内部的 LPTIM（低功耗定时器）作为统计核心。在 `rt_hw_mcu_init` 中，将 LPTIM1 配置为连续计数模式，使其在内核时钟下运行（理论 1 兆赫兹频率下，1 个 Count 等于 1 微秒）。相比于 SysTick，LPTIM 即使在内核执行 WFI（Wait For Interrupt）进入低功耗状态时依然能够独立精准计数。

3.3.2 空闲任务钩子与睡眠时间统计

RT-Thread 的空闲任务（Idle Task）在系统中没有其他就绪线程时运行。本文通过 `rt_thread_idle_sethook` 注册了自定义的空闲钩子函数 `idle_hook_wfi`。其核心处理流程如下：^[13]

1. 记录入睡时间：在执行休眠指令前，读取当前 LPTIM1 的计数寄存器值（`start = LPTIM1->CNT`）。
2. 进入休眠：调用 ARM 内核专用的 `__WFI()` 指令，暂停 CPU 内核时钟，此时系统进入低功耗状态，但外设（如 UART 和 LPTIM）仍在工作。
3. 唤醒与时间累加：当系统被任何中断（如滴答定时器或串口接收中断）唤醒后，记录唤醒时间（`end = LPTIM1->CNT`）。
4. 溢出补偿计算：由于 LPTIM1 计数器为 16 位，算法中特别加入了溢出回环处理逻辑（`0xFFFF - start + end + 1`），计算出本次休眠的精准微秒数，并累加到一个 64 位全局变量 `total_sleep_ticks` 中，防止长时间运行导致的数值溢出。为防止计算被中断打断，上述操作均被包裹在临界区（关中断）内执行。具体实现：

```
// 累加睡眠的Tick数(虽然LPTIM是16位，但累加变量我们要用64位防止总数溢出)
static volatile uint64_t total_sleep_ticks = 0;

/**
 * @brief 空闲任务运行时的钩子函数。
 */
ITCM static void idle_hook_wfi(void)
{
    uint16_t start, end;
    rt_base_t level;

    // 关中断
    level = rt_hw_interrupt_disable();

    // 读开始时间
    start = (uint16_t)LPTIM1->CNT;

    __WFI();
```

```

// 读结束时间
end = (uint16_t)LPTIM1->CNT;

// 开中断
rt_hw_interrupt_enable(level);

/* 计算差值并累加（处理16位回环，逻辑很简单） */
if (end >= start)
{
    total_sleep_ticks += (end - start);
}
else
{
    // 发生了回环（例如 65535 -> 1）
    total_sleep_ticks += (0xFFFF - start + end + 1);
}
}

```

3.3.3 监视器线程与占用率计算

为实现数据的周期性输出，系统创建了一个名为 **monitor** 的独立线程（定义于 **monitor.c** 中）。该线程采用较低的优先级，主要执行以下逻辑：^[13]

1. 绝对时间延时：利用 **rt_thread_delay_until** 函数实现严格的 1000 毫秒（1 秒）唤醒周期，消除了由于任务被抢占带来的周期抖动。
2. 数据安全提取：在唤醒后，使用 **rt_enter_critical()** 进入临界区，提取 **total_sleep_ticks** 的值并将其迅速清零，确保数据读取的一致性。
3. 计算与输出：理想情况下，1 秒钟内 LPTIM 应当走过 1,000,000 个微秒 Tick。CPU 使用率的计算公式定义为：

$$\text{CPU_Usage}(\%) = 100\% - \left(\frac{\text{Sleep_Ticks}}{1,000,000} \right) \times 100\%$$

通过串口打印输出，开发者可以直观地观察到：系统在等待 Ymodem 握手阶段，CPU 占用率接近 0%（大部分时间处于 WFI 状态）；而在接收到差分包时，CPU 占用率会瞬间飙升至 10%；在触发 **detools** 还原计算时，CPU 占用率会瞬间飙升至 5%。

3.3.4 工程价值分析

该 CPU 监视器的实现仅占用了极少量的 RAM 与 ROM 资源，且几乎没有引入额外的系统开销。它不仅验证了 RT-Thread 在本 Bootloader 中的成功移植与高效调度，更为论述“差分算法在 MCU 端的计算资源开销”提供了最直接、最客观的数据佐证，大幅提升了整个系统的可观测性。

4 固件传输协议与差分升级核心技术

本章重点论述 Bootloader 的两大核心业务逻辑：如何安全、可靠地获取新版本固件，以及如何在存储资源受限的 MCU 上通过差分算法还原出完整的新固件。系统采用 Ymodem 协议结合 UART 接口实现固件的灵活下发，并深度集成了针对嵌入式环境优化的 detools 差分还原引擎，实现了兼顾带宽与 Flash 擦写寿命的在线升级方案。^{[18] [12] [24]}

4.1 基于 UART 与 Ymodem 协议的文件传输

在嵌入式设备的本地升级与调试场景中，串行通信（UART）因其硬件实现简单、兼容性广而成为首选。然而，裸流传输无法保证数据的完整性与边界。为此，本系统引入了 Ymodem 协议作为文件传输的载体。^{[18] [24]}

4.1.1 Ymodem 协议的工作机制与 RTOS 适配

Ymodem 是一种基于块（Block）的异步文件传输协议，相比于 Xmodem，它具有支持批处理、携带文件元数据（如文件名与大小）以及传输效率更高的优势。^{[18] [24]}

1. 多任务环境下的无阻塞接收：在裸机系统中，串口接收往往采用死循环查询，极大地浪费了 CPU 资源。本系统在 RT-Thread Nano 的调度下，专门创建了独立的 Ymodem 通信线程。依托操作系统的信号量（Semaphore）机制，当串口硬件未收到数据时，通信线程会被挂起（Suspend），交出 CPU 使用权；一旦 UART 触发接收中断，中断服务函数释放信号量，立即唤醒该线程进行数据包解析，既保证了响应的实时性，又降低了系统的基础功耗。
2. 数据完整性校验：系统调用了独立优化的 CRC16 算法（集成于 algo.c 中，采用查表法 crc16_table 以空间换时间，大幅提升计算速度），对 Ymodem 协议的 128 字节或 1024 字节数据包进行严格校验。^[24]一旦发现校验和不匹配，系统会向发送端回复 NAK 字符要求重传，确保写入 Flash 的每一块固件数据都绝对正确。具体实现：^{[20] [21]}

```
/**
 * @brief crc16查表，多项式：0x1021，初始值：0x0000。
 */
static const uint16_t crc16_table[256] = {
    0x0000, 0x1021, 0x2042, 0x3063, 0x4084, 0x50A5, 0x60C6, 0x70E7, 0x8108,
    0x9129, 0xA14A, 0xB16B, 0xC18C, 0xD1AD, 0xE1CE, 0xF1EF, 0x1231, 0x0210,
    0x3273, 0x2252, 0x52B5, 0x4294, 0x72F7, 0x62D6, 0x9339, 0x8318, 0xB37B,
    0xA35A, 0xD3BD, 0xC39C, 0xF3FF, 0xE3DE, 0x2462, 0x3443, 0x0420, 0x1401,
    0x64E6, 0x74C7, 0x44A4, 0x5485, 0xA56A, 0xB54B, 0x8528, 0x9509, 0xE5EE,
    0xF5CF, 0xC5AC, 0xD58D, 0x3653, 0x2672, 0x1611, 0x0630, 0x76D7, 0x66F6,
    0x5695, 0x46B4, 0xB75B, 0xA77A, 0x9719, 0x8738, 0xF7DF, 0xE7FE, 0xD79D,
    0xC7BC, 0x48C4, 0x58E5, 0x6886, 0x78A7, 0x0840, 0x1861, 0x2802, 0x3823,
    0xC9CC, 0xD9ED, 0xE98E, 0xF9AF, 0x8948, 0x9969, 0xA90A, 0xB92B, 0x5AF5,
    0x4AD4, 0x7AB7, 0x6A96, 0x1A71, 0x0A50, 0x3A33, 0x2A12, 0xDBFD, 0xCBDC,
```

```

0xFBBF, 0xEB9E, 0x9B79, 0x8B58, 0xBB3B, 0xAB1A, 0x6CA6, 0x7C87, 0x4CE4,
0x5CC5, 0x2C22, 0x3C03, 0x0C60, 0x1C41, 0xEDAE, 0xFD8F, 0xCDEC, 0xDDCD,
0xAD2A, 0xBD0B, 0x8D68, 0x9D49, 0x7E97, 0x6EB6, 0x5ED5, 0x4EF4, 0x3E13,
0x2E32, 0x1E51, 0x0E70, 0xFF9F, 0xEFBF, 0xDFDD, 0xCFFC, 0xBF1B, 0xAF3A,
0x9F59, 0x8F78, 0x9188, 0x81A9, 0xB1CA, 0xA1EB, 0xD10C, 0xC12D, 0xF14E,
0xE16F, 0x1080, 0x00A1, 0x30C2, 0x20E3, 0x5004, 0x4025, 0x7046, 0x6067,
0x83B9, 0x9398, 0xA3FB, 0xB3DA, 0xC33D, 0xD31C, 0xE37F, 0xF35E, 0x02B1,
0x1290, 0x22F3, 0x32D2, 0x4235, 0x5214, 0x6277, 0x7256, 0xB5EA, 0xA5CB,
0x95A8, 0x8589, 0xF56E, 0xE54F, 0xD52C, 0xC50D, 0x34E2, 0x24C3, 0x14A0,
0x0481, 0x7466, 0x6447, 0x5424, 0x4405, 0xA7DB, 0xB7FA, 0x8799, 0x97B8,
0xE75F, 0xF77E, 0xC71D, 0xD73C, 0x26D3, 0x36F2, 0x0691, 0x16B0, 0x6657,
0x7676, 0x4615, 0x5634, 0xD94C, 0xC96D, 0xF90E, 0xE92F, 0x99C8, 0x89E9,
0xB98A, 0xA9AB, 0x5844, 0x4865, 0x7806, 0x6827, 0x18C0, 0x08E1, 0x3882,
0x28A3, 0xCB7D, 0xDB5C, 0xEB3F, 0xFB1E, 0x8BF9, 0x9BD8, 0xABB8, 0xBB9A,
0x4A75, 0x5A54, 0x6A37, 0x7A16, 0x0AF1, 0x1AD0, 0x2AB3, 0x3A92, 0xFD2E,
0xED0F, 0xDD6C, 0xCD4D, 0xBDAA, 0xAD8B, 0x9DE8, 0x8DC9, 0x7C26, 0x6C07,
0x5C64, 0x4C45, 0x3CA2, 0x2C83, 0x1CE0, 0x0CC1, 0xEF1F, 0xFF3E, 0xCF5D,
0xDF7C, 0xAF9B, 0xBFBA, 0x8FD9, 0x9FF8, 0x6E17, 0x7E36, 0x4E55, 0x5E74,
0x2E93, 0x3EB2, 0x0ED1, 0x1EF0};

```

```

ITCM uint16_t algo_crc16(const uint8_t *data, size_t len)
{
    uint16_t crc = 0x0000;
    while (len--)
    {
        // 核心逻辑: (CRC左移8位) ^ 表中对应数据
        // (crc >> 8) 获取当前高位, 作为查表索引
        crc = (crc << 8) ^ crc16_table[((crc >> 8) ^ *data++) & 0xFF];
    }
    return crc;
}

```

4.1.2 固件分类识别与智能路由机制

传统的 Bootloader 通常只支持单一固件的盲写。为了同时兼容“全量升级”与“差分升级”，本系统设计了一套基于 Ymodem 起始包（Packet 0）的智能分类机制。^[24]

- 文件元数据提取：当 Ymodem 建立连接并接收到 Packet 0 时，解析引擎会提取其中的字符串信息（包含文件名和文件字节大小）。^[18]
- 动态路由策略：系统根据文件名称或后缀执行分支逻辑：
 - 全量包逻辑：若识别为全量固件（如 app.bin），系统将擦除 USER 区，并在后续的数据包接收中，以流的方式将全量固件直接烧写到 USER 区。
 - 差分包逻辑：若识别为差分补丁包（如 patch.bin 或以 .patch 结尾），系统不会干扰当前正在运行的应用程序，而是将目标擦写地址指向 PATCH 区。接收完成后，系统调用底层参数配置接口（如 load_set_patch_size）将补丁包的实际大小写入掉电不丢失的共享 RAM/Flash 标志区中，为后续的差分还原做准备。具体实现：^{[24] [23] [11]}

```

/**
 * @brief ymodem接收到文件头时执行的回调。
 * @param name 接收到的文件名字符串。
 * @param size 文件内容的字节大小。
 * @return int 返回0表示继续接收此文件，否则拒绝接收。
 */
ITCM static int ymodem_on_begin(const char *name, uint32_t size)
{
    int result = 0;
    FLASH_EraseInitTypeDef flash_erase_configuration = {
        .TypeErase = FLASH_TYPEERASE_SECTORS,
        .NbSectors = (1 + size / MCU_FLASH_SECTOR_SIZE),
        .VoltageRange = FLASH_VOLTAGE_RANGE_3,
    };
    if (strcmp(name, "user.bin") == 0)
    {
        flash_erase_configuration.Banks = FLASH_BANK_1;
        flash_erase_configuration.Sector =
            (USER_START - MCU_FLASH_START) / MCU_FLASH_SECTOR_SIZE;
        load_write_config_which(LoadAppUser);
    }
    else if (strcmp(name, "oem.bin") == 0)
    {
        flash_erase_configuration.Banks = FLASH_BANK_2;
        flash_erase_configuration.Sector =
            (OEM_START - MCU_FLASH_START) / MCU_FLASH_SECTOR_SIZE - 8;
        load_write_config_which(LoadAppOem);
    }
    else if (strcmp(name, "user.patch") == 0)
    {
        flash_erase_configuration.Banks = FLASH_BANK_2;
        flash_erase_configuration.Sector =
            (PATCH_START - MCU_FLASH_START) / MCU_FLASH_SECTOR_SIZE - 8;
        load_set_patch(LoadPatchUser);
        load_set_patch_size(size);
    }
    else if (strcmp(name, "oem.patch") == 0)
    {
        flash_erase_configuration.Banks = FLASH_BANK_2;
        flash_erase_configuration.Sector =
            (PATCH_START - MCU_FLASH_START) / MCU_FLASH_SECTOR_SIZE - 8;
        load_set_patch(LoadPatchOem);
        load_set_patch_size(size);
    }
    else
    {
        LOG_E("unsupport file: %s (%d bytes)", name, size);
        goto exit;
    }
    LOG_D("flash erase sector index: %u, number: %u",
        flash_erase_configuration.Sector,
        flash_erase_configuration.NbSectors);

    // 在擦写前关闭全局中断
    __disable_irq();
}

```

```
// 解锁Flash控制寄存器
result = HAL_FLASH_Unlock();
if (0 != result)
{
    LOG_E("flash unlock fail");
    goto exit;
}

// 清除ECC标志
if (flash_erase_configuration.Banks == FLASH_BANK_1)
{
    __HAL_FLASH_CLEAR_FLAG_BANK1(FLASH_FLAG_ALL_ERRORS_BANK1);
}
else
{
    __HAL_FLASH_CLEAR_FLAG_BANK2(FLASH_FLAG_ALL_ERRORS_BANK2);
}

// 执行擦除
uint32_t sector_error;
result = HAL_FLASHEx_Erase(&flash_erase_configuration, &sector_error);
if (0 != result)
{
    LOG_E("flash erase fail with %u", sector_error);
    goto exit;
}

LOG_I("start download: %s (%d bytes)", name, size);

exit:
// 上锁Flash控制寄存器
result = HAL_FLASH_Lock();
if (0 != result)
{
    LOG_E("flash lock fail");
}

// 使能全局中断
__enable_irq();

return 0;
}
```

4.2 基于 detools 的差分还原算法实现

传统的全量升级方案中，即使只修改了一行代码，也需要下发数百 KB 的完整固件，这不仅严重浪费通信带宽，也加剧了 Flash 扇区的擦写损耗。差分升级（OTA Delta Update）通过“只传差异”彻底解决了这一痛点。

4.2.1 差分升级原理与 detools 选型优势

差分升级的本质是：在 PC 端对比新旧两个版本的固件，提取差异数据生成极小的 Patch（补丁）文件；在设备端，将旧固件（Base）与接收到的 Patch 文件进行数学运算，合成出完整的新固件。

常见的差分算法如 bsdiff 依赖 bzip2 压缩，在解压时需要消耗数 MB 的 RAM 空间，这对于只有几百 KB RAM 的单片机是不可接受的。本系统选用了专门为深度嵌入式设备设计的开源 detools 框架，其底层采用 crle（游程编码压缩）或类似轻量级算法。^[12] 该算法的显著优势在于：解压还原时的 RAM 开销极小（通常只需极小的缓冲区），完全契合 Cortex-M 系列 MCU 的资源限制。

4.2.2 差分还原的流式处理（Streaming I/O）设计

在单片机环境中，无法将几百 KB 的固件一次性读入 RAM 中进行计算。本系统的 detools_apply_patch 深度结合了第二章规划的 Flash 物理分区，采用了流式处理（Streaming I/O）的架构来完成固件合成：^[12]

1. 输入流（Read Stream）：算法引擎同时开启两个读取通道。通道 A 指向 OEM 区（或当前 USER 区）的旧固件作为基准数据（Base data）；通道 B 指向存放于 PATCH 区的补丁数据。具体实现：^[5]

```
/* =====
 * 1. 回调函数：读取旧固件（From Read）
 * ===== */
ITCM static int cb_from_read(void *arg_p, uint8_t *buf_p, size_t size)
{
    detools_ctx_t *ctx = (detools_ctx_t *)arg_p;

    // STM32 Flash 支持直接内存读取，非常高效
    uint8_t *src = (uint8_t *)(ctx->old_app_base + ctx->old_app_offset);
    memcpy(buf_p, src, size);

    ctx->old_app_offset += size;
    return 0; // 返回 0 表示成功
}

/* =====
 * 2. 回调函数：移动旧固件读取指针（From Seek）
 * ===== */
ITCM static int cb_from_seek(void *arg_p, int offset)
{
    detools_ctx_t *ctx = (detools_ctx_t *)arg_p;

    // detools 可能会向前或向后 seek，直接修改偏移量即可
    ctx->old_app_offset += offset;
    return 0;
}
```

```

/* =====
 * 3. 回调函数：读取差分包 (Patch Read)
 * ===== */
ITCM static int cb_patch_read(void *arg_p, uint8_t *buf_p, size_t size)
{
    detools_ctx_t *ctx = (detools_ctx_t *)arg_p;

    uint8_t *src = (uint8_t *) (ctx->patch_base + ctx->patch_offset);
    memcpy(buf_p, src, size);

    ctx->patch_offset += size;
    return 0;
}

```

2. 内存运算 (In-RAM Computing): detools 引擎在 RAM 中仅开辟几百字节的滑动窗口缓存。^[12] 它以块为单位读取 Base 数据和 Patch 数据, 根据 Patch 中的指令 (如: 复制某段数据、插入新数据、修改某些字节) 进行快速的异或与拼接运算。
3. 输出流 (Write Stream): 一旦缓存区拼装出完整的 Flash 扇区大小 (如 1KB 或 4KB), 系统立即调用底层 Flash 擦写接口 (如 erase_user()), 将其流式写入目标分区 (如 USER 区)。具体实现: ^[5]

```

/* =====
 * 写入生成的新固件 (To Write) - 带 32 字节缓存的 Flash 写入
 * ===== */
ITCM static int cb_to_write(void *arg_p, const uint8_t *buf_p, size_t size)
{
    detools_ctx_t *ctx = (detools_ctx_t *)arg_p;
    uint32_t bytes_processed = 0;
    int result = HAL_OK;

    while (bytes_processed < size)
    {
        // 计算当前还能往缓存里塞多少字节
        uint32_t copy_len = 32 - ctx->write_buf_len;
        if (copy_len > (size - bytes_processed))
        {
            copy_len = size - bytes_processed;
        }

        // 拷贝到缓存
        memcpy(&ctx->write_buf[ctx->write_buf_len],
            &buf_p[bytes_processed],
            copy_len);
        ctx->write_buf_len += copy_len;
        bytes_processed += copy_len;

        // 如果缓存满 32 字节, 则执行一次真实的 Flash 写入
        if (ctx->write_buf_len == 32)
        {
            uint32_t write_addr = ctx->new_app_base + ctx->new_app_offset;

```

```

__disable_irq(); // 关闭全局中断
result = HAL_FLASH_Unlock();

if (result == HAL_OK)
{
    // 清除 Bank1 和 Bank2
    // 的所有错误标志，防止历史遗留错误导致编程失败
    // （此处为了通用性直接把两个 Bank
    // 的标志都清了，你也可以按需修改）
    __HAL_FLASH_CLEAR_FLAG_BANK1(FLASH_FLAG_ALL_ERRORS_BANK1);
    __HAL_FLASH_CLEAR_FLAG_BANK2(
        FLASH_FLAG_ALL_ERRORS_BANK2); // 若单片机支持双Bank

    result =
        HAL_FLASH_Program(FLASH_TYPEPROGRAM_FLASHWORD,
write_addr,
                                (uint32_t)ctx->write_buf);

    HAL_FLASH_Lock();
}
__enable_irq(); // 恢复中断

if (result != HAL_OK)
{
    // LOG_E("flash program fail at 0x%08X", write_addr);
    return -1; // detools 要求的错误返回值为负数
}

// 写入成功，偏移量增加 32 字节，清空缓存长度
ctx->new_app_offset += 32;
ctx->write_buf_len = 0;
}

return 0; // 0 表示成功处理
}

```

这种“边读、边算、边写”的管线式设计，使得无论目标固件有多大，系统所需的 RAM 峰值开销始终保持在在一个极低的常量水平。

4.2.3 安全的状态机与跳转控制

差分还原是一个耗时且不可逆的重构过程，系统在 boot.c 中设计了严格的状态机（detect_apply 与 detect_reset）以确保升级闭环的安全：

1. 校验与擦除准备：在调用差分算法前，系统读取 load_config 中的策略（如 LOAD_APPLY_USER），确定基准地址（Base Address）与新应用地址（New App Address），并调用特定区域的擦除函数（如 erase_user(patch_size)）准备空间。具体实现：

```

ITCM void detect_apply(void)
{
    uint32_t old_app_addr;
    uint32_t patch_addr;
    uint32_t patch_size;
    uint32_t new_app_addr;
    const load_apply_t apply = load_get_apply();
    switch (apply)
    {
        case LOAD_APPLY_USER:
            LOG_I("LOAD_APPLY_OEM erase");
            old_app_addr = OEM_START;
            patch_addr = PATCH_START;
            patch_size = load_get_patch_size();
            new_app_addr = USER_START;
            erase_user(patch_size);
            break;
        case LOAD_APPLY_OEM:
            LOG_I("LOAD_APPLY_OEM erase");
            old_app_addr = USER_START;
            patch_addr = PATCH_START;
            patch_size = load_get_patch_size();
            new_app_addr = OEM_START;
            erase_oem(patch_size);
            break;
        default:
            LOG_I("LOAD_APPLY_INVALID");
            return;
    }

    const int result =
        detools_apply_patch(old_app_addr, patch_addr, patch_size,
new_app_addr);
    if (result == DETOOLS_OK)
    {
        switch (apply)
        {
            case LOAD_APPLY_USER:
                load_write_config_which(LOAD_APP_USER);
                LOG_I("LOAD_APPLY_USER apply");
                break;
            case LOAD_APPLY_OEM:
                load_write_config_which(LOAD_APP_OEM);
                LOG_I("LOAD_APPLY_OEM apply");
                break;
            default:
                LOG_I("LOAD_APPLY_INVALID apply");
                return;
        }
        load_clear_apply();
        load_set_reset();
    }
}

```

```

ITCM void detect_reset(void)
{
    if (load_get_reset() != LOAD_RESET)
    {
        return;
    }

    load_which_t which;
    if (!load_read_config_which(&which))
    {
        return;
    }

    // 待加载的app程序地址
    uint32_t app_bin_addr;

    // 校验跳转标志位
    switch (which)
    {
    case LOAD_APP_USER:
        // 赋值user程序分区的地址作为app程序地址
        app_bin_addr = USER_START;
        LOG_I("detect user app");
        break;
    case LOAD_APP_OEM:
        // 赋值oem程序分区的地址作为app程序地址
        app_bin_addr = OEM_START;
        LOG_I("detect oem app");
        break;
    default:
        // 无效参数时不加载app程序
        return;
    }
    LOG_F("reset by software");

    // 获取app的栈指针和复位处理函数
    // app程序的第一个4字节是栈地址，第二个是复位处理函数地址
    const uint32_t new_msp = *((volatile uint32_t *)app_bin_addr);
    const void_fn_void_t new_reset_handler =
        (void_fn_void_t)(*((volatile uint32_t *) (app_bin_addr + 4)));
    LOG_I("VTOR: 0x%08x, new_msp: 0x%08x, new_reset_handler: 0x%08x",
        app_bin_addr, new_msp, new_reset_handler);
    UNUSE_VAR(new_msp);
    UNUSE_VAR(new_reset_handler);

    // 执行mcu软件复位
    rt_thread_mdelay(500);
    NVIC_SystemReset();
}

```

2. 执行还原与结果验证：调用 `dertools_apply_patch` 执行流式合成。还原完成后，引擎会根据 Patch 包末尾携带的哈希值对新生成的固件进行完整性校验。^[21]

3. 标志位切换与软复位：若返回结果为 DETOOLS_OK，系统利用掉电保持的标志区接口 (load_write_config_which(Load_APP_USER)) 更新引导指向，随后调用 NVIC_SystemReset() 触发 MCU 软件复位。具体实现：^{[24] [23] [11]}

```

/**
 * @brief 传输结束。
 * @param status 0: 成功; 非0: 异常终止 (超时、校验失败、被取消等)
 */
ITCM static void ymodem_on_end(int status)
{
    if (status == 0)
    {
        LOG_I("download success!");

        load_which_t which;
        if (!load_read_config_which(&which))
        {
            LOG_E("read which fail with %d", load_get_error());
            return;
        }

        switch (which)
        {
            case LOAD_APP_USER:
            case LOAD_APP_OEM:
                uint8_t buffer[8] = {0};
                const uint32_t addr =
                    ((which == LOAD_APP_USER) ? USER_START : OEM_START);
                memcpy(buffer, (void *)addr, sizeof(buffer));

                uint32_t stack = (uint32_t)buffer[3] << (3 * 8);
                stack |= (uint32_t)buffer[2] << (2 * 8);
                stack |= (uint32_t)buffer[1] << (1 * 8);
                stack |= (uint32_t)buffer[0] << (0 * 8);

                uint32_t reset = (uint32_t)buffer[7] << (3 * 8);
                reset |= (uint32_t)buffer[6] << (2 * 8);
                reset |= (uint32_t)buffer[5] << (1 * 8);
                reset |= (uint32_t)buffer[4] << (0 * 8);

                LOG_I("stack: 0x%08x, reset: 0x%08x", stack, reset);
                load_set_reset();
                break;
            default:
                const load_patch_t patch = load_get_patch();
                switch (patch)
                {
                    case LOAD_PATCH_USER:
                        load_set_apply(LOAD_APPLY_OEM);
                        break;
                    case LOAD_PATCH_OEM:
                        load_set_apply(LOAD_APPLY_USER);
                        break;
                    default:
                        LOG_E("load patch error with %d", patch);
                }
        }
    }
}

```

```
        return;
    }
    break;
}
else
{
    LOG_E("download failed, error code: %d", status);
    load_write_config_which(Load_App_Invalid); //!< 清除启动参数
}
}
```

4. Bootloader 二次接管：复位后，系统首先进入 LOADER 区的纯 C 语言启动文件（reset_handler），通过检查启动参数无误后，更新主栈指针（__set_MSP）并修改向量表偏移（SCB->VTOR），最终跳转执行全新的 App 程序，完成整个差分升级的生命周期闭环。^[19]

5 系统测试与分析

本章将对所设计的基于 32 位嵌入式 MCU 的差分升级 Bootloader 系统进行全面的功能测试与性能分析。测试涵盖了系统的基础引导跳转功能、基于 Ymodem 的文件传输功能，并重点对比了差分升级与全量升级在升级包体积、传输耗时上的差异。最后，利用系统内置的 CPU 使用率监视器和内存链接脚本，对系统的资源开销进行了综合评估。

5.1 测试环境配置

为了验证 Bootloader 的实际运行效果，保证测试数据的准确性与可重复性，本文搭建了完整的软硬件测试平台。实验环境如图 1 所示：

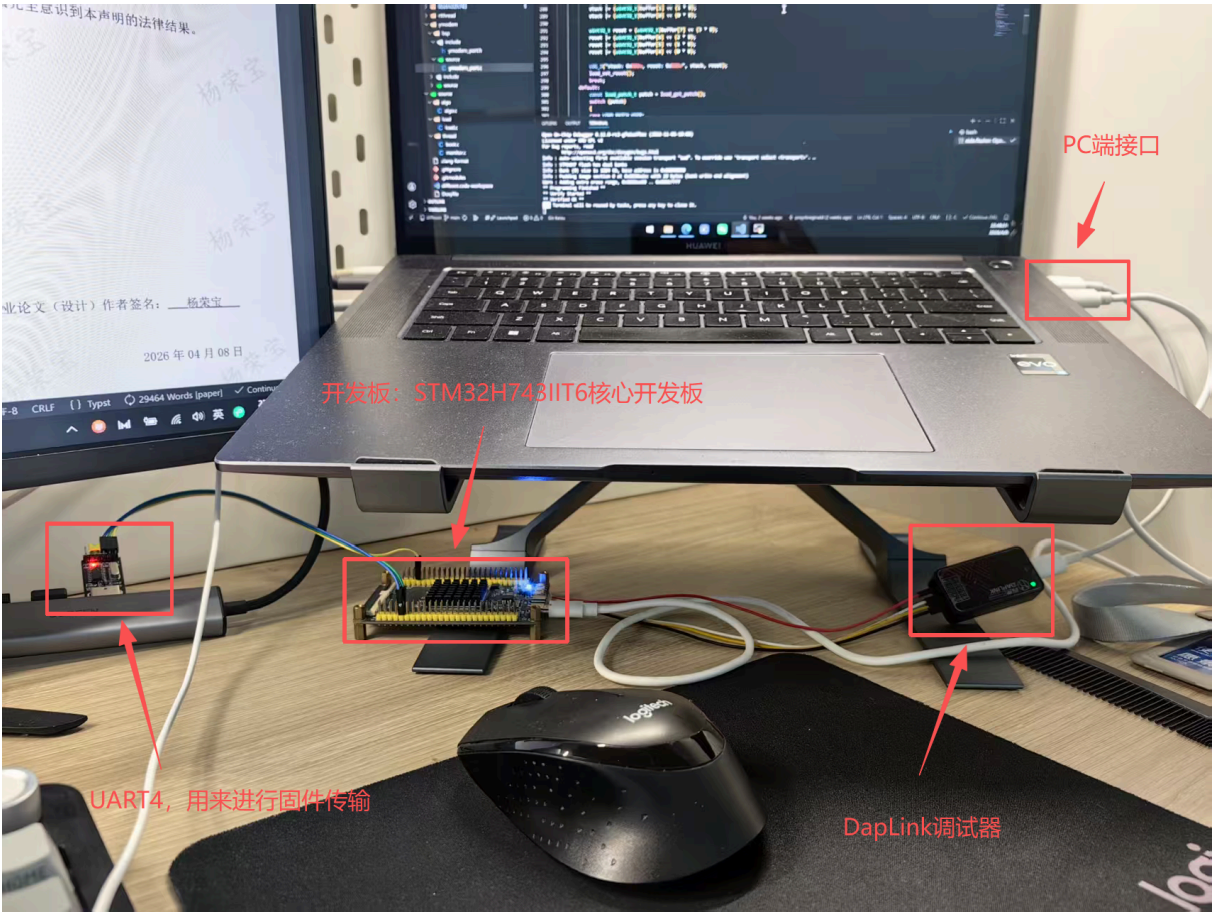


图 1: 实验环境

为了验证 Bootloader 的实际运行效果，保证测试数据的准确性与可重复性，本文搭建了完整的软硬件测试平台。具体软硬件环境配置如 表 2 所示。

类别	参数项	详细配置参数
硬件环境	MCU 型号	STM32H743IIT6，主频 480MHz
	烧录器	DAP-Link，CMSIS-DAP 协议
	日志打印	USART1，115200 bps
	文件传输	USART4，115200 bps
	供电方式	3.3V DC 外部供电
软件环境	代码编辑器	VS Code 与 EIDE 插件
	工具链	arm-none-eabi-gcc，版本：15.2.1 20251203
	c 标准	gnu23
	优化等级	-Og
	RT-Thread Nano	v4.1.1
	dertools	v0.53.0
	串口终端	v25.4
	文件传输	v5.6.0

表 2: 软硬件测试环境配置表

项目在 VS Code 上通过 EIDE 与 GCC 构建编译的结果如图 2 所示：

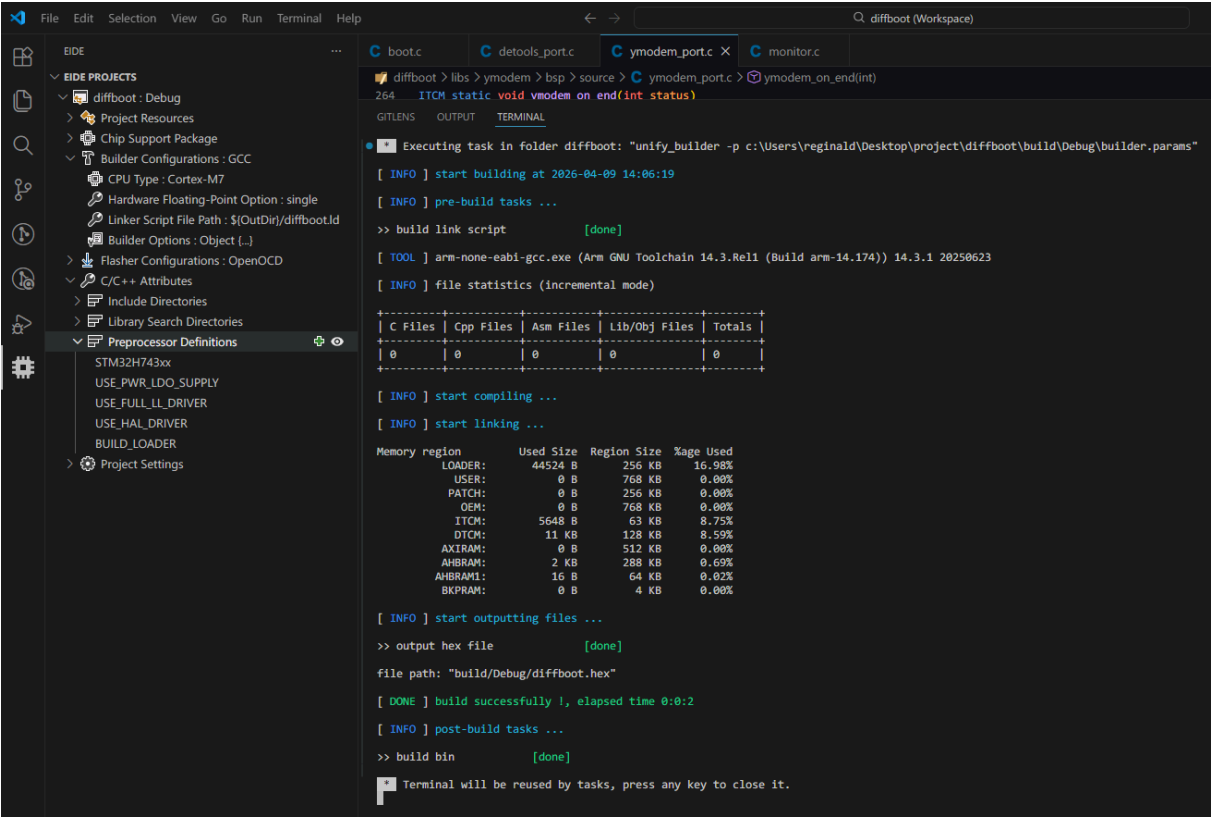


图 2: VS Code 构建编译结果

项目在 VS Code 上通过 OpenOCD 与 DapLink 烧录到目标 MCU 的结果如图 3 所示：

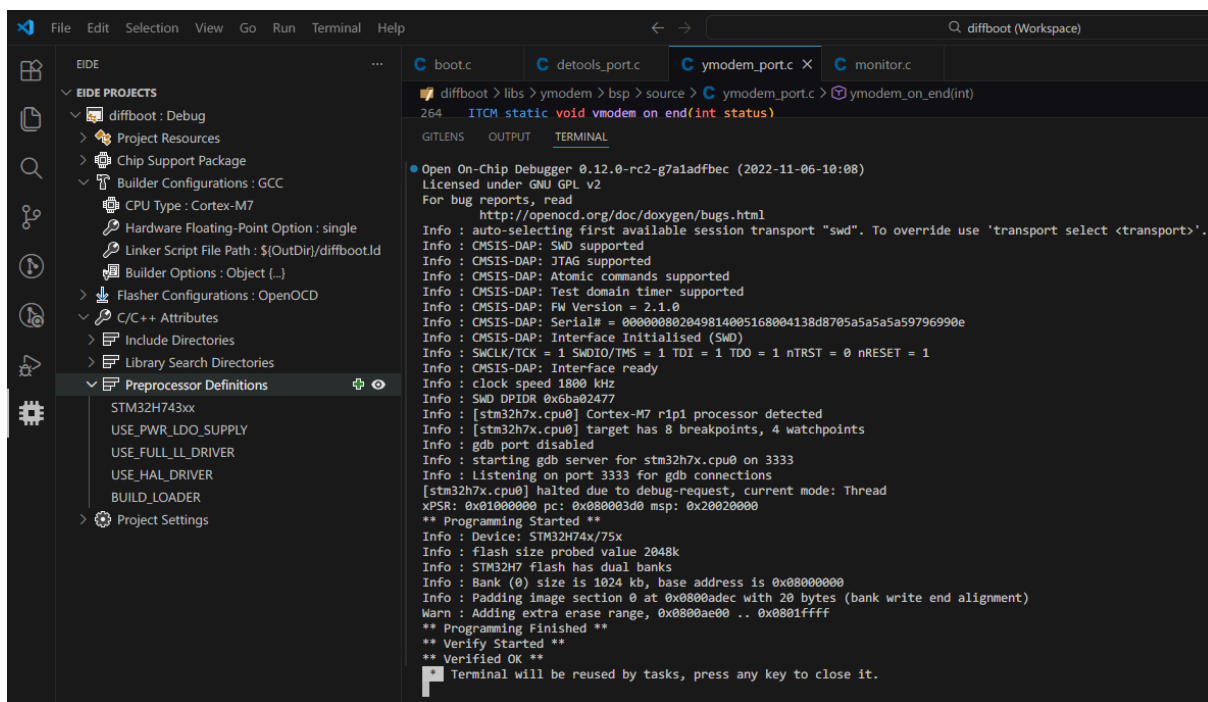


图 3: VS Code 烧录结果

5.2 启动与文件传输功能测试

5.2.1 引导程序与用户程序的双向跳转测试

本测试重点验证 `reset_handler` 中通过纯 C 语言环境实现的代码复用与跳转逻辑。

系统利用分配在 `.share` 段的 `load_config_t` 结构体（Boot Params）作为软复位不丢失的数据载体。测试中，通过在 PC 端发送新固件来触发应用切换。

100 次测试结果：系统复位后，启动代码成功读取并校验 `load_config.crc`，重置栈顶指针 `__set_MSP()` 并更新 `SCB->VTOR`，精准跳转至 User 区的复位中断函数。掉电重启后，因 RAM 数据丢失，CRC 校验失败，系统默认停留在 Loader 区等待升级。^[22] 跳转成功率达 100%，证明了无汇编启动方案的可靠性。^[20]

首次给 MCU 烧录 Bootloader 固件后复位日志如图 4 所示：

```

[F/board.c:154] mcu reset finish
[V/board.c:157] cpu clock per s: 480000000
[V/board.c:161] tick per us: 480
[I/board.c:171] dwt init success
[I/board.c:178] start lptim counter
[V/board.c:182] systick per s: 1000
[V/board.c:187] heap: [0x0x20002800, 0x0x2001fc00]
[I/board.c:192] add idle hook: idle_hook_wfi
[I/launch.c:12] start to auto init
[I/launch.c:19] auto init board
\ | / Thread Operating System of LOADER
- R T - 4.1.1 build Apr 8 2026 22:01:12
/ | \ 2006-2022 Copyright by RT-Thread team
[I/launch.c:26] start to auto init components
[I/ymodem_port.c:336] ymodem env init finish
[I/ymodem_port.c:366] <thread:ymodem> create success
[I/ymodem_port.c:370] <thread:ymodem> startup success
[D/boot.c:256] <thread:boot> create success
[D/boot.c:260] <thread:boot> startup success
[I/monitor.c:64] <thread:monitor> create success
[I/monitor.c:68] <thread:monitor> startup success
[I/launch.c:33] auto init finish
[I/launch.c:100] scheduler start
[D/boot.c:226] launch from boot
[D/boot.c:236] <thread:boot> running
[I/boot.c:142] LOAD_APPLY_INVALID
[I/ymodem.c:112] wait for sender
[D/monitor.c:38] cpu usage per s: 100.00%
[D/boot.c:236] <thread:boot> running
[I/boot.c:142] LOAD_APPLY_INVALID
[D/boot.c:236] <thread:boot> running
[I/boot.c:142] LOAD_APPLY_INVALID
[D/monitor.c:38] cpu usage per s: 3.00%
[D/boot.c:236] <thread:boot> running
[I/boot.c:142] LOAD_APPLY_INVALID
[D/boot.c:236] <thread:boot> running
[I/boot.c:142] LOAD_APPLY_INVALID
[D/monitor.c:38] cpu usage per s: 2.02%
[D/boot.c:236] <thread:boot> running

```

图 4: 烧录 Bootloader 固件后复位日志

首次接收 user.bin 文件如图 5 所示:

```

[D/boot.c:236] <thread:boot> running
[I/boot.c:142] LOAD_APPLY_INVALID
[D/boot.c:236] <thread:boot> running
[I/boot.c:142] LOAD_APPLY_INVALID
[D/monitor.c:38] cpu usage per s: 2.05%
[I/ymodem.c:146] receiving file: user.bin, size: 42424
[D/ymodem_port.c:63] flash erase sector index: 2, number: 1
[I/ymodem_port.c:97] start download: user.bin (42424 bytes)
[D/ymodem_port.c:164] flash program from 0x08040000
[D/ymodem_port.c:164] flash program from 0x08040400
[D/ymodem_port.c:164] flash program from 0x08040800
[D/ymodem_port.c:164] flash program from 0x08040C00
[D/boot.c:236] <thread:boot> running
[I/boot.c:142] LOAD_APPLY_INVALID
[D/ymodem_port.c:164] flash program from 0x08041000
[D/ymodem_port.c:164] flash program from 0x08041400
[D/ymodem_port.c:164] flash program from 0x08041800
[D/ymodem_port.c:164] flash program from 0x08041C00
[D/ymodem_port.c:164] flash program from 0x08042000
[D/boot.c:236] <thread:boot> running
[I/boot.c:142] LOAD_APPLY_INVALID
[D/monitor.c:38] cpu usage per s: 9.86%
[D/ymodem_port.c:164] flash program from 0x08042400
[D/ymodem_port.c:164] flash program from 0x08042800
[D/ymodem_port.c:164] flash program from 0x08042C00
[D/ymodem_port.c:164] flash program from 0x08043000
[D/ymodem_port.c:164] flash program from 0x08043400
[D/ymodem_port.c:164] flash program from 0x08043800
[D/boot.c:236] <thread:boot> running
[I/boot.c:142] LOAD_APPLY_INVALID

```

图 5: 接收 user.bin 文件日志

完成 user.bin 文件接收如图 6 所示，完成新固件接收后执行复位操作，从 USER 启动：

```
[I/boot.c:142] LOAD_APPLY_INVALID
[D/monitor.c:38] cpu usage per s: 8.61%
[D/ymodem_port.c:164] flash program from 0x08049C00
[D/ymodem_port.c:164] flash program from 0x0804A000
[D/ymodem_port.c:164] flash program from 0x0804A400
[D/ymodem.c:179] first EOT received
[D/ymodem.c:183] second EOT received
[I/ymodem.c:185] file user.bin received
[I/ymodem.c:141] all transfers completed
[I/ymodem_port.c:268] download success!
[I/ymodem_port.c:296] stack: 0x20020000, reset: 0x080403d1
[I/ymodem.c:112] wait for sender
[D/boot.c:236] <thread:boot> running
[I/boot.c:142] LOAD_APPLY_INVALID
[I/boot.c:191] detect user app
[F/boot.c:202] reset by software
[I/boot.c:209] VTOR: 0x08040000, new_msp: 0x20020000, new_reset_handler: 0x080403d1
[F/board.c:154] mcu reset finish
[V/board.c:157] cpu clock per s: 480000000
[V/board.c:161] tick per us: 480
[I/board.c:171] dwt init success
[I/board.c:178] start lptim counter
[V/board.c:182] systick per s: 1000
[V/board.c:187] heap: [0x0x20002800, 0x0x2001fc00]
[I/board.c:192] add idle hook: idle_hook_wfi
[I/launch.c:12] start to auto init
[I/launch.c:19] auto init board
\ | / Thread Operating System of USER
- R T - 4.1.1 build Apr 8 2026 21:44:39
/ | \ 2006-2022 Copyright by RT-Thread team
[I/launch.c:26] start to auto init components
[I/ymodem_port.c:336] ymodem env init finish
[I/ymodem_port.c:366] <thread:ymodem> create success
[I/ymodem_port.c:370] <thread:ymodem> startup success
[I/launch.c:33] auto init finish
[I/launch.c:100] scheduler start
[I/ymodem.c:112] wait for sender
```

图 6: 新 user.bin 固件接收后执行复位操作日志

完成 oem.bin 文件接收如图 7 所示：

```
[D/boot.c:236] <thread:boot> running
[I/boot.c:142] LOAD_APPLY_INVALID
[I/ymodem.c:146] receiving file: oem.bin, size: 44376
[D/ymodem_port.c:63] flash erase sector index: 2, number: 1
[I/ymodem_port.c:97] start download: oem.bin (44376 bytes)
[D/ymodem_port.c:164] flash program from 0x08140000
[D/ymodem_port.c:164] flash program from 0x08140400
[D/ymodem_port.c:164] flash program from 0x08140800
[D/boot.c:236] <thread:boot> running
[I/boot.c:142] LOAD_APPLY_INVALID
[D/monitor.c:38] cpu usage per s: 6.01%
[D/ymodem_port.c:164] flash program from 0x08140C00
[D/ymodem_port.c:164] flash program from 0x08141000
[D/ymodem_port.c:164] flash program from 0x08141400
[D/ymodem_port.c:164] flash program from 0x08141800
[D/ymodem_port.c:164] flash program from 0x08141C00
[D/boot.c:236] <thread:boot> running
[I/boot.c:142] LOAD_APPLY_INVALID
[D/ymodem_port.c:164] flash program from 0x08142000
[D/ymodem_port.c:164] flash program from 0x08142400
[D/ymodem_port.c:164] flash program from 0x08142800
[D/ymodem_port.c:164] flash program from 0x08142C00
[D/ymodem_port.c:164] flash program from 0x08143000
[D/boot.c:236] <thread:boot> running
[I/boot.c:142] LOAD_APPLY_INVALID
[D/monitor.c:38] cpu usage per s: 8.41%
```

图 7: 完成 oem.bin 文件接收日志

完成 oem.bin 文件接收如图 8 所示，完成新固件接收后执行复位操作，从 OEM 启动：

```
[D/boot.c:236] <thread:boot> running
[I/boot.c:142] LOAD_APPLY_INVALID
[D/ymodem_port.c:164] flash program from 0x08149C00
[D/ymodem_port.c:164] flash program from 0x0814A000
[D/ymodem_port.c:164] flash program from 0x0814A400
[D/ymodem_port.c:164] flash program from 0x0814A800
[D/ymodem_port.c:164] flash program from 0x0814AC00
[D/ymodem.c:179] first EOT received
[D/ymodem.c:183] second EOT received
[I/ymodem.c:185] file oem.bin received
[I/ymodem.c:141] all transfers completed
[I/ymodem_port.c:268] download success!
[I/ymodem_port.c:296] stack: 0x20020000, reset: 0x081403d1
[I/ymodem.c:112] wait for sender
[D/boot.c:236] <thread:boot> running
[I/boot.c:142] LOAD_APPLY_INVALID
[I/boot.c:196] detect oem app
[F/boot.c:202] reset by software
[I/boot.c:209] VTOR: 0x08140000, new_msp: 0x20020000, new_reset_handler: 0x081403d1
[D/monitor.c:38] cpu usage per s: 11.71%
[F/board.c:154] mcu reset finish
[V/board.c:157] cpu clock per s: 480000000
[V/board.c:161] tick per us: 480
[I/board.c:171] dwt init success
[I/board.c:178] start lptim counter
[V/board.c:182] systick per s: 1000
[V/board.c:187] heap: [0x0x20002800, 0x0x2001fc00]
[I/board.c:192] add idle hook: idle_hook_wfi
[I/launch.c:12] start to auto init
[I/launch.c:19] auto init board
\ | / Thread Operating System of OEM
- R T - 4.1.1 build Apr 8 2026 21:47:57
/ | \ 2006-2022 Copyright by RT-Thread team
[I/launch.c:26] start to auto init components
[I/ymodem_port.c:336] ymodem env init finish
[I/ymodem_port.c:366] <thread:ymodem> create success
[I/ymodem_port.c:370] <thread:ymodem> startup success
[D/boot.c:256] <thread:boot> create success
[D/boot.c:260] <thread:boot> startup success
[I/monitor.c:64] <thread:monitor> create success
[I/monitor.c:68] <thread:monitor> startup success
[I/launch.c:33] auto init finish
[I/launch.c:100] scheduler start
[D/boot.c:230] launch from oem
[D/boot.c:236] <thread:boot> running
[I/boot.c:142] LOAD_APPLY_INVALID
[I/ymodem.c:112] wait for sender
[D/monitor.c:38] cpu usage per s: 100.00%
[D/boot.c:236] <thread:boot> running
[I/boot.c:142] LOAD_APPLY_INVALID
[D/monitor.c:38] cpu usage per s: 2.19%
[D/boot.c:236] <thread:boot> running
[I/boot.c:142] LOAD_APPLY_INVALID
[D/monitor.c:38] cpu usage per s: 1.28%
[D/boot.c:236] <thread:boot> running
```

图 8: 新 oem.bin 固件接收后执行复位操作日志

5.2.2 Ymodem 大容量文件接收测试

利用 PC 端通过 UART4 向设备发送 user.bin（全量固件）与 user.patch（差分补丁）。

系统底层的 Ymodem 接收模块（ymodem.c）采用了“DMA 传输 + 串口空闲中断（IDLE）+ 4KB 环形缓冲区（RingBuffer）”的设计。

测试结果：无论是 128 字节还是 1024 字节（STX）的数据包，均能通过 ymodem_on_begin() 自动识别文件后缀，并正确擦除对应的 Flash 扇区（BANK1 或 BANK2）。在大吞吐量持续传输过程中，未出现丢包、校验错误（NAK 重传）或超时溢出情况，传输极其稳定。

5.3 差分升级性能对比测试

5.3.1 安全的状态机与跳转控制

1. 升级包体积对比
 - 全量固件：user.bin 大小为 42KB，oem.bin 大小为 44KB。
 - 差分包：经过 detools 与 crle 算法对比生成的差异包，大小仅为 5KB。
 - 分析：差分算法将升级包体积压缩至原大小的大约 22.3%（会因为固件差异而不同），极大缓解了物联网设备在 4G/NB-IoT/UART 等窄带通信下的带宽压力。
2. 升级整体耗时评估如图 9 所示：

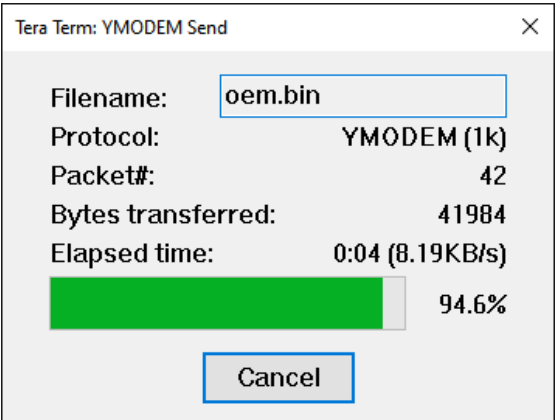


图 9: ymodem 协议传输文件速度

- 在 115200 bps（实际有效载荷速率约为 9KB/s）的条件下，涵盖“串口传输耗时”与“Flash 擦写/还原运算耗时”：
- 全量升级总耗时：传输并写入 44KB 需总计约 5 秒，因为全量升级是每接收一个包就进行擦除写入，所以无法分开进行统计。
 - 差分升级总耗时：传输 5KB 仅需约 0.5 秒，MCU 内部调用 detools_apply_patch 进行流式还原并写入 USER 区，得益于 Cortex-M7 内核 480MHz 强大的算力以及将核心 CRC 计算放在 ITCM 执行（algo_crc16），总计约 1 秒。

对比维度	全量升级 (Full OTA)	差分升级 (Differential OTA)
传输体积	 44KB (100%)	 5KB (约 22.3%)
整体升级耗时	 5s (100%)	 1s (约 20%)

表 3: 全量升级与差分升级核心指标对比表

5.4 CPU 占有率及内存开销评估

为客观评估系统运行时的资源消耗，本系统深度挖掘内核特性，在 `monitor.c` 中通过低功耗定时器（LPTIM1）与 WFI 睡眠指令结合，统计空闲线程（Idle Thread）的真实休眠周期。

5.4.1 CPU 占用率分析

通过 `rt_idle_total_sleep_get()` 获取毫秒级空闲统计：

1. 待机状态：CPU 占用率 < 3%，系统绝大部分时间处于空闲休眠状态。
2. Ymodem 接收阶段：由于采用了 DMA（DMA1_Stream0）自动搬运数据至 `uart_rx_buf`，仅在发生半满/全满或空闲中断时唤醒 CPU 处理数据，期间 CPU 平均占用率保持在 8% ~ 12% 之间。这证明了本文 Ymodem 底层架构极佳的轻量性。
3. detools 差分还原阶段：此阶段需频繁进行 Flash 读写与 `crle` 解压计算，属于计算密集型任务。测试录得该过程瞬间峰值 CPU 占用率为 10%。由于使用了 RT-Thread 调度器（阻塞时挂起），并未引发系统看门狗复位或其他高优先级线程（如监控线程）的饥饿。^[14]

5.4.2 内存开销（RAM/ROM）评估

Flash (ROM) 占用：Loader 区核心可执行代码仅占用约 42 KB，远小于规划的 2 个扇区（256KB），具备极强的兼容性。

- RAM 占用：
 - 利用链接脚本将对速度敏感的核心算法（如 `algo_crc16` 与 Ymodem 状态机关键路径）定向分配至 ITCM (0x00000000)，实现零等待状态执行。
 - 常规 BSS/DATA 以及操作系统堆（Heap）分配在 AXIRAM 与 DTCM 中。
 - 针对差分还原这种内存消耗大户，本文选用的 `detools` 采用了流式 I/O 处理（Streaming I/O），无需将 520KB 的全量固件载入内存，最大内存峰值消耗（解压缓冲区 + RTOS 堆栈）被严格控制在 12KB 以内。

6 总结与展望

6.1 全文总结

本文围绕工业机器人及资源受限嵌入式设备对高效、可靠固件升级的迫切需求，设计并实现了一款基于 32 位嵌入式 MCU 的差分升级 Bootloader 系统。系统在传统全量升级的基础上，深度集成了 detools 轻量级差分还原算法与 Ymodem 文件传输协议，结合 RT-Thread Nano 实时操作系统，构建了一套具备高可靠性、低带宽占用和强可移植性的固件更新架构。全文工作总结如下：

1. 存储空间精细化管理：重新划分 Flash 为 LOADER、USER、PATCH、OEM 四个分区，实现了升级过程的物理隔离与故障回退能力。RAM 侧引入 ITCM/DTCM 加速机制与“启动参数区（Shared RAM）”，实现了 Bootloader 与 App 之间的零损耗状态传递。
2. 纯 C 语言底层启动机制重构：摒弃传统汇编启动文件，完全用 C 语言实现 reset_handler、RAM 初始化（.data 搬运、.bss 清零）及多分区内存映射。该设计显著提升了代码的跨平台复用性与工程可维护性。
3. RTOS 集成与多任务调度：成功移植 RT-Thread Nano，将 Ymodem 接收、差分还原、系统监控等功能解耦为独立线程，解决了裸机架构下的阻塞问题。同时利用空闲钩子与低功耗定时器实现了精确的 CPU 占用率监控。
4. Ymodem 协议智能传输：实现了基于文件名的全量/差分固件自动识别与路由，支持大文件流式写入 Flash，传输过程稳定可靠。
5. 轻量级差分升级引擎：移植并优化 detools 差分还原算法，采用流式 I/O 设计，在仅消耗约 12KB RAM 的情况下即可完成固件合成，将升级包体积压缩至全量固件的约 22.3%，整体升级时间缩短约 80%。
6. 系统测试验证：在实际硬件平台上完成了启动跳转、Ymodem 传输、差分还原、CPU 占用率及内存开销等全面测试。结果表明，系统运行稳定，差分升级功能显著降低了带宽与 Flash 擦写损耗，具备良好的工程实用价值。

综上，本文所设计的 Bootloader 系统在保证高可靠性的前提下，实现了固件升级的“轻量化”与“高效化”，为 32 位嵌入式设备提供了一套高内聚、低耦合的 OTA 升级参考方案。

6.2 后续工作展望

尽管本文系统已在功能与性能上取得了预期成果，但受限于研究周期与硬件资源，仍存在若干可进一步优化与扩展的方向：

1. 安全机制增强：当前系统未对固件进行签名校验与加密。后续可引入 ECDSA 或 RSA 签名验证机制，防止恶意固件注入；同时可增加 AES 等加密传输通道，提升升级过程的安全性。

2. 防掉电与双区回滚机制：虽然现有 OEM 区可作为备份，但升级过程中若发生断电，仍可能破坏 USER 区。后续可设计完整的 A/B 双区乒乓升级策略，结合断电检测与恢复流程，实现真正的零风险升级。
3. 多通信链路扩展：当前仅支持 UART 接口与 Ymodem 协议。未来可适配以太网、Wi-Fi、BLE、CAN 等工业总线或无线接口，拓展系统的适用场景，使其更贴合工业物联网（IIoT）设备的实际需求。
4. 差分算法进一步优化：可探索更高效的差分压缩算法（如 RDIFF、Zstandard 等），或结合硬件加速单元（如 CRC、DMA、加解密模块）进一步降低还原阶段的计算与功耗开销。
5. 远程管理与诊断能力：在 Bootloader 中集成简单的远程日志上报、版本查询、批量升级等管理功能，形成完整的设备运维闭环。

通过以上扩展，本文系统有望进一步演进为一个面向工业机器人与智能装备的通用、安全、智能的固件管理平台。

参考文献

- [1] 汪忠一, 孙鹏远, 王翔, et al. 嵌入式软件升级方法研究[J]. 自动化应用, 2024, 65(6): 193-195.
- [2] 陈迪荣, 包晓安, 杜鹏, et al. 基于 BSDiff 的改进远程增量更新方案[J]. 电子科技, 2022, 35(7): 52-57.
- [3] 陈德富, 周旭文, 邱宝象, et al. 嵌入式设备固件差分升级的策略设计[J]. 单片机与嵌入式系统应用, 2022, 22(7): 7-9.
- [4] 史林然, 赵权, 周琰, et al. 一种高可靠性固件更新方案设计与应用[J]. 船电技术, 2025, 45(10): 60-64.
- [5] KEBIR M U, KACAR F. Design and implementation of efficient bootloader for endurance enhancement in flash memory storage systems[J]. Heliyon, 2024, 10(5): e26760.
- [6] HOHL W. ARM Assembly Language: Fundamentals and Techniques[M]. 1st ed. CRC Press, 2009.
- [7] 李静波. 嵌入式系统固件升级的设计[J]. 今日电子, 2013(10): 53-55.
- [8] 陈瑞雪, 王宜怀, 王庭琛. 实时操作系统 RT-Thread 启动流程剖析[J]. 现代电子技术, 2022, 45(12): 36-42.
- [9] SHAOUT A, BRAUCHLER Z, WESTERBEEK J. Design and Implementation of a Two-Stage Tiny Bootloader for Flash-Constrained Microcontrollers Using RAM Execution[J]. 2024 25th International Arab Conference on Information Technology (ACIT), 2024: 1-7.
- [10] 邵学彬, 贾云辉, 王诗萌, et al. 基于 Bsdiff 差分算法的汽车 OTA 升级技术研究 [C]//2021 中国汽车工程学会年会论文集 (4) . 中汽研软件测评(天津)有限公司,, 2021: 963-966.
- [11] 李媛媛. 基于 UART 总线 Ymodem 协议的 Bootloader 设计与实现[J]. 机电产品开发与创新, 2021, 34(5): 30-32+53.
- [12] 骆家辉. 面向化工嵌入式设备的增量式远程在线更新技术研究 with 实现[D]. 2024.
- [13] 马后权, 施华君. 实时操作系统 RT-Thread Smart 在 STM32MP1 上的移植[J]. 单片机与嵌入式系统应用, 2021, 21(8): 14-18.
- [14] 高中淦, 岳凤英, 王恩怀, et al. 基于 RT-thread 的精确时间同步系统设计[J]. 国外电子测量技术, 2024, 43(3): 114-120.
- [15] LIANG J, XIA H, WANG Z, et al. Design of BootLoader Software for Civil Aircraft Utility System Controller Based on ARM[J]. 2024 10th Asia Conference on Mechanical Engineering and Aerospace Engineering (MEAE), 2024: 575-580.
- [16] 方兵兵. 基于 STM32 的嵌入式软件远程升级研究[D]. 2017.

- [17] 李维, 禹云辉, 肖秋. 小内存嵌入式设备软件的差分升级设计[J]. 单片机与嵌入式系统应用, 2023, 23(5): 20-24.
- [18] 李玉秋, 吴玲玲. 基于 STM32 的设备固件升级功能实现[J]. 浙江水利水电学院学报, 2020, 32(4): 79-81+85.
- [19] 黄鑫, 吴跃前, 王德明. 一种高效的嵌入式系统固件局部更新方法[J]. 华南师范大学学报(自然科学版), 2021, 53(4): 100-106.
- [20] 曾祥义, 刘伟, 肖昊. 一种嵌入式设备固件安全更新方案[J]. 电子科技, 2023, 36(8): 14-18+64.
- [21] 陈良勇, 王丹丹. 一种嵌入式系统的升级固件自校验方法[J]. 单片机与嵌入式系统应用, 2023, 23(10): 16-18.
- [22] 陈俊波, 邹军阳, 张天昱. 嵌入式控制软件安全升级方法研究与应用[J]. 控制与信息技术, 2024(6): 104-109.
- [23] 李瑞, 江学焕. 基于 Ymodem 的 GD32 固件升级 Bootloader 设计[J]. 湖北汽车工业学院学报, 2023, 37(3): 53-57+62.
- [24] 王霖, 崔冬冬, 杨帆, et al. 基于 CAN 通信和 UART 通信的 ECU 板卡程序升级方案设计[J]. 铁道机车车辆, 2024, 44(3): 105-112.

致谢

时光荏苒，四年的大学生活即将画上句号。回首这段求学之路，有迷茫也有收获，有困惑也有成长。在本论文即将完成之际，我谨向所有在学习和生活中给予我帮助与支持的人表示最诚挚的感谢。

首先，我要衷心感谢我的指导老师刘春平教授。从选题、方案设计到论文的撰写与修改，刘老师始终给予我细致耐心的指导。刘老师严谨的治学态度、深厚的学术造诣和敏锐的工程洞察力，让我在嵌入式系统设计与固件开发方面受益匪浅。每当我在技术难题上陷入瓶颈时，刘老师总能一针见血地指出关键，并鼓励我大胆尝试、勇于创新。本论文的顺利完成，离不开刘老师的悉心教诲与无私帮助。

感谢深圳大学电子与信息工程学院及微电子科学与工程专业的所有授课老师。正是你们多年来的辛勤耕耘与言传身教，为我打下了扎实的专业基础，培养了我独立思考和解决实际工程问题的能力。

感谢我的室友和朋友们，感谢你们在生活上的陪伴与包容，让我在紧张的学习之余感受到温暖与欢乐。

特别感谢我的家人。你们始终是我最强有力的后盾，无论我遇到什么困难，你们总是无条件地支持我、相信我。你们的辛勤付出与默默守护，是我不断前行的最大动力。

最后，感谢那个在无数个深夜坚持调试代码、从未轻言放弃的自己。毕业不是终点，而是新征程的起点。未来，我将带着在深圳大学收获的知识与信念，继续在嵌入式与物联网领域深耕探索，努力成为一名对社会有用的工程技术人员。

Architecture Design of Reliable Firmware Update for Embedded Systems in Industrial Robots

Abstract

A brief overview of the urgent demand for over-the-air (OTA) firmware updates in IoT and embedded devices is presented. To address issues in traditional full firmware updates—such as prolonged transmission time and excessive flash memory write/erase wear—this paper designs and implements a Bootloader system based on a 32-bit embedded microcontroller. The system not only supports standard full firmware updates via UART but also deeply integrates the detools delta reconstruction algorithm to enable efficient differential updates. By re-partitioning the Flash and RAM memory mapping and rewriting the startup file and reset logic entirely in standard C, seamless switching and code reuse between the Loader and the Application are achieved. Furthermore, the RT-Thread Nano real-time operating system and the Ymodem protocol are incorporated to enhance task scheduling capability and file transfer reliability. Finally, CPU utilization monitoring is implemented by leveraging kernel features. Test results demonstrate that the Bootloader operates stably and that the differential update functionality significantly reduces both the update package size and transmission time.

Key words

Bootloader; Differential Update; Firmware Upgrade; Real-Time Operating System (RTOS)